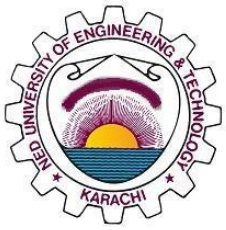
Machine Vision Pipeline for Steel Surface Defects Detection using FPGA SoC

UNDERGRADUATE FINAL YEAR PROJECT REPORT

Department of Computer & Information Systems Engineering
NED University of Engineering and Technology



Group Number: 33
Batch: 2021 – 2022

Group Member Names:
Aaliyan Mansoor CS-21146
Muhammad Babar Ali CS-21145
Mohib Ud Din CS-21142
Muhammad Maaz Khan CS-21019

Approved by

_____

Miss Fakhra Aftab
Lecturer
Project Advisor

THIS PAGE IS INTENTIONALLY LEFT BLANK

Author's Declaration

We declare that we are the sole authors of this project report. It is the actual copy of the project that was accepted by our advisor(s) including any necessary revisions. We also grant NED University of Engineering and Technology permission to reproduce and distribute electronic or paper copies of this project report.

Signatures and Date

_____	Aaliyan Mansoor	CS-21146
_____	Muhammad Babar Ali	CS-21145
_____	Mohib Ud Din	CS-21142
_____	Muhammad Maaz Khan	CS-21019

Statement of Contributions

Each student contributed to the project as follows:

- **Aaliyan Mansoor**: Performed model inspection and quantization, data preprocessing (AgriAutos dataset), UI and Integration.
- **Muhammad Babar Ali**: Performed model inspection and quantization, model training and optimization, UI and Integration.
- **Mohib Ud Din**: Performed data preprocessing (NEUDET+GC10DET), model training and optimization, FPGA Deployment.
- **Muhammad Maaz Khan**: Performed data preprocessing (NEUDET), model inspection and quantization, FPGA Deployment.

Literature review, report writing, and research paper was done collectively by all members.

Executive Summary

This project report provides a comprehensive overview of the Machine Vision Pipeline for Steel Surface Defects Detection using FPGA SoC. The primary objective is to address challenges in automated defect detection in the steel industry by employing an FPGA-based system.

The report covers the design implementation and evaluation of the project. A state of the art CNN(Convolutional Neural Network) YOLO-V5 was trained on two datasets, publicly available datasets NEUDET and GC10DET were combined to improve model performance and a dataset from the industry (AgriAutos).

The model was quantized and compiled using Vitis-AI, a framework developed by Xilinx. The model was deployed on the Ultra96-V2 which is a ARM based Zynq UltraScale+ MPSoC. DPU pynq is used to utilize the capabilities of the DPU(Deep Processing Unit) of the Programmable fabric of the board for inference of the deployed model and offload the postprocessing tasks to the ARM based CPU of the board.

Comprehensive testing was conducted on benchmark datasets and AgriAutos dataset. The system achieved high detection accuracy of upto 0.98 on GPU, 0.93 after quantization and 0.85 on the board. The mean power consumption of the board ranged from 4.7W to 5.7W with idle power consumption at 3.055W.

This work shows the potential effectiveness of FPGA based machine vision acceleration. For industrial applications this provides a framework for deploying machine vision models in industrial settings. This paves the way for further extensions and research in this domain, improving the performance of the overall system and adding additional features to facilitate industries.

Acknowledgments

We would like to express our deepest gratitude to Miss Fakhra Aftab, supervisor and Dr. Majida Kazmi, co-supervisor, for their invaluable guidance, support, and encouragement throughout this project. We also thank the faculty and staff of the Department of Computer & Information Systems Engineering for their assistance.

Special thanks to the administration of NED University for providing the necessary resources and support to make this project possible.

Table of Contents

Author's Declaration ii
Statement of Contributions iii
Executive Summary iv
Acknowledgments v
List of Figures ix
List of Abbreviations x
United Nations Sustainable Development Goals xi

1 Introduction **1**
1.1 Background Information 1
1.2 Significance and Motivation 1
1.3 Aims and Objectives 2
1.4 Methodology 2
1.5 Machine Vision Pipeline Diagram 3
1.6 Report Outline 3

2 Literature Review **5**
2.1 Introduction 5
2.2 Machine Vision in Steel Surface Inspection 5
2.2.1 EC-YOLO: Effectual Detection Model for Steel Strip Surface Defects Based on YOLO-V5 5
2.2.2 Lightweight Surface Defect Detection Algorithm Based on Improved YOLOv5 6
2.2.3 LCG-YOLO: A Real-Time Surface Defect Detection Method for Metal Components 6
2.2.4 Metal Surface Defect Detection using Object Detection Models 6
2.2.5 An Enhanced YOLOv4 Model with Self-dependent Attentive Fusion and Component Randomized Mosaic Augmentation for Metal Surface Defect Detection 6
2.3 FPGA Applications in Machine Learning 7
2.3.1 FPGA-Based Real-Time Object Detection and Classification System Using YOLO for Edge Computing 7
2.3.2 Design of a YOLO Model Accelerator Based on PYNQ Architecture 7
2.3.3 Design of Low-power YOLO Accelerator Based on FPGA . . . 8
2.3.4 Design and Implementation of the High-performance YOLO Accelerator based on Zynq FPGA 8
2.3.5 Resource-constrained FPGA implementation of YOLOv2 . . . 8

2.3.6 A Power-Efficient Optimizing Framework FPGA Accelerator Based on Winograd for YOLO . . . 9
2.4 Gap Analysis . . . 9

3 Data Processing **11**
3.1 Introduction . . . 11
3.2 Dataset Overview . . . 11
3.3 Data Preprocessing . . . 13
3.3.1 Dataset Preparation . . . 13
3.3.2 Data Annotation . . . 13
3.3.3 Data Augmentation . . . 14
3.3.4 Splitting Dataset . . . 16
3.3.5 Conversion and Exporting to YOLOv5-Compatible Format . . . 16

4 Model and Results **17**
4.1 Introduction . . . 17
4.2 Model Selection . . . 17
4.2.1 Comparing YOLOv3 and YOLOv5 . . . 18
4.3 Model Architecture . . . 18
4.3.1 CSP Backbone . . . 19
4.3.2 PANet Neck . . . 20
4.3.3 Activation Function . . . 21
4.4 Model Optimization and Training . . . 22
4.5 Results and Analysis . . . 23
4.5.1 Performance Metrics . . . 23
4.5.2 Results . . . 23
4.5.3 Visualization of Results . . . 24
4.6 Discussion . . . 31

5 FPGA Deployment **32**
5.1 Introduction . . . 32
5.2 FPGA Board . . . 32
5.3 Vitis AI Deployment Workflow . . . 33
5.4 Model Inspection . . . 34
5.5 Model Preparation . . . 35
5.6 Quantization . . . 36
5.6.1 Vitis AI Quantizer Workflow . . . 37
5.6.2 Quantization Techniques . . . 38
5.6.3 Quantization Process . . . 39
5.7 Compiling . . . 40
5.7.1 Overview . . . 40
5.7.2 Methodology . . . 40
5.8 DPU-PYNQ . . . 42
5.8.1 Overview . . . 42
5.8.2 Hardware and Software Stack . . . 44
5.8.3 Software Flow for Model Deployment . . . 46
5.8.4 Architecture of the DPU . . . 49
5.8.5 Complete Workflow Explanation . . . 51
5.8.6 Deployment on Ultra96-V2 (Case Study) . . . 52

5.8.7 Conclusion 55
5.9 Discussion 55

6 Results **57**
6.1 Overview 57
6.2 Quantitative Performance Metrics 57
6.3 Detection Results Visualization 57
6.4 Performance Optimization 60
6.5 Discussion 60

7 User Interface **61**
7.1 Overview 61
7.2 Features and Functionality 62
7.3 Technology Stack 63
7.4 Impact and Role in the Pipeline 63

8 Conclusion and Future Enhancements **65**
8.1 Conclusion 65
8.2 Future Enhancements 66
8.3 Real-time Defect Detection with Camera Integration 66
8.3.1 Dual AR0144 IAS Sensor Modules 66
8.3.2 Single AR1335 IAS Sensor Module 67
8.3.3 Enhanced Energy Efficiency with Advanced FPGA Platform . . 68
8.3.4 Expanded Defect Classification and Model Refinement 68
8.3.5 Backend Integration and Industrial Dashboard 68
8.4 Closing Remarks 69

List of Figures

1.1 Machine Vision Pipeline for Steel Surface Defect Detection. 3

3.1 NEU-DET dataset defects 12
3.2 Defects on steel rods from Agriauto dataset 13
3.3 Annotated images of NEU-DET(a) and Agriautos(b) Dataset using Roboflow 14
3.4 Dataset Augmentation Settings in Roboflow 14

4.1 Yolov5 Model Architecture. 19
4.2 Confusion Matrix on NEUDET and GC10 Dataset 24
4.3 Confusion Matrix on AgriAutos Dataset 25
4.4 F1 cruve on NEUDET and GC10 Dataset 26
4.5 PR (Precision-Recall) curve on NEUDET and GC10 Dataset 27
4.6 F1 scores AgriAuto Dataset 28
4.7 PR (Precision-Recall) on AgriAuto Dataset 29
4.8 NEUDET+GC10 inference images on test set 30
4.9 AgriAuto test image inference 31

5.1 Ultra96-V2 Architecture. 33
5.2 Vitis AI Deployment Workflow. 34
5.3 Relu vs LeakyRelu. 35
5.4 Quantizer Workflow in Vitis AI. 37
5.5 Quantization with Vitis AI. 38
5.6 Compiling with Vitis AI. 41
5.7 PYNQ DPU Architecture 43
5.8 ultra96v2 board architecture diagram 45
5.9 Software Flow 48
5.10 DPU Micro-architecture 49
5.11 Ultra96v2 Inference Flow 54

6.1 FPGA Inference. 58
6.2 FPGA Inference. 59

7.1 SteelSight Home Page: Overview and Project Summary 62
7.2 Detection Page: Upload Interface and Real-Time Metrics 63

8.1 OnSemi Dual Camera Mezzanine Board for 96Boards 67
8.2 Dual AR0144 IAS Sensor Modules 67
8.3 Single AR1335 IAS Sensor Modules 68

List of Abbreviations

CPU	Central Processing Unit
GPU	Graphics Processing Unit
DPU	Deep Processing Unit
PL	Programmable Logic
PS	Processing System
FPGA	Field-Programmable Gate Array
ReLU	Rectified Linear Unit
Leaky ReLU	Leaky Rectified Linear Unit
YOLO	You Only Look Once
mAP	Mean Average Precision
FPS	Frames Per Second
TP	True Positive
FP	False Positive
FN	False Negative
CLB	Configurable Logic Block
LUT	Look-Up Table
MPSoC	Multiprocessor System-on-Chip
DSP	Digital Signal Processor
CMT	Clock Management Tile
RAM	Random Access Memory
DDR	Double Data Rate
INT	Integer Format
FPU	Floating Point Unit
ALU	Arithmetic Logic Unit
NNDCT	Neural Network Deployment Compiler Tool
CNN	Convolutional Neural Network
PTQ	Post-Training Quantization
QAT	Quantization-Aware Training
CSS	Cascading Style Sheets
JS	JavaScript
AI	Artificial Intelligence
ML	Machine Learning
UI	User Interface
NMS	Non-max suppression
ARM	Advanced RISC Machine
RISC	Reduced Instruction Set Computer

United Nations Sustainable Development Goals

The Sustainable Development Goals (SDGs) are the blueprint to achieve a better and more sustainable future for all. They address the global challenges we face, including poverty, inequality, climate change, environmental degradation, peace and justice. The following checked SDGs are related to our project:

- ☐ **No Poverty**
- ☐ **Zero Hunger**
- ☐ **Good Health and Well being**
- ☐ **Quality Education**
- ☐ **Gender Equality**
- ☐ **Clean Water and Sanitation**
- ☐ **Affordable and Clean Energy**
- ☑ **Decent Work and Economic Growth**
- ☑ **Industry, Innovation and Infrastructure**
- ☐ **Reduced Inequalities**
- ☐ **Sustainable Cities and Communities**
- ☐ **Responsible Consumption and Production**
- ☐ **Climate Action**
- ☐ **Life Below Water**
- ☐ **Life on Land**
- ☐ **Peace and Justice and Strong Institutions**
- ☑ **Partnerships to Achieve the Goals**

Chapter 1

Introduction

1.1 Background Information

Steel surfaces, such as steel rods and hot-rolled steel strips, are integral to industries like automotive, aerospace, manufacturing, and construction. However, these surfaces are prone to defects such as cracks, inclusions, scratches, and pitted surfaces, which compromise quality, safety, and operational efficiency. Identifying such defects early in the manufacturing process is critical for reducing waste, minimizing rework costs, and ensuring the integrity of end products.[4][29]

Traditional inspection methods rely on manual or conventional automated systems that struggle to meet the precision and speed demands of modern industrial environments. Although machine vision-based deep learning models offer a promising solution, their deployment on CPUs or GPUs is constrained by slower processing speeds and higher energy consumption. FPGAs, with their parallel processing capabilities and low-power requirements, present a viable alternative for deploying real-time machine vision systems.

This project proposes a novel approach to defect detection by integrating a deep learning model into an FPGA System on Chip (SoC) for steel surface inspection. The proposed solution aims to deliver high accuracy, real-time processing, and energy efficiency, addressing key challenges in industrial automation and quality control.

1.2 Significance and Motivation

The significance of this project lies in its potential to revolutionize steel surface defect detection by combining the precision of deep learning with the efficiency of FPGA technology. While existing solutions often compromise between speed, accuracy, and

energy efficiency, this project aims to balance these factors to create a scalable, real-time system for industrial applications.

As industries increasingly prioritize automation, energy efficiency, and quality assurance, this project aligns with the evolving demands of smart manufacturing. The proposed FPGA-based pipeline not only enhances defect detection but also demonstrates the feasibility of deploying advanced AI solutions in resource-constrained environments, setting a benchmark for future advancements in industrial automation.

1.3 Aims and Objectives

The primary aim of our project is to design and implement a machine vision pipeline for real-time steel surface defect detection using an FPGA SoC. The key objectives include:

- Develop a robust deep learning model for detecting various steel surface defects.
- Preprocess datasets to optimize model training and evaluation.
- Deploy the trained model on an FPGA SoC for high-speed, low-power inference.
- Evaluate performance metrics, including accuracy, latency, and energy efficiency.
- Compare the FPGA-accelerated solution with traditional CPU and GPU-based systems.
- Align the solution with industry standards to support future scalability and adoption.

1.4 Methodology

The methodology adopted in this project is systematic and iterative, ensuring the achievement of the project's objectives. The key steps are:

1. **Problem Analysis:** Identify challenges in defect detection and define requirements for an FPGA-based solution.
2. **Dataset Preparation:** Collect, preprocess, and annotate datasets to train the machine vision pipeline.
3. **Model Selection and Training:** Research, design, and train a deep learning model suitable for defect detection.

4. **FPGA Deployment:** Optimize and deploy the trained model on an FPGA SoC, addressing hardware constraints through techniques like quantization and model inspection.

5. **Performance Evaluation:** Test the system for speed, accuracy, and energy efficiency in real-world conditions.

6. **Comparative Analysis:** Benchmark the FPGA-based solution against conventional CPU and GPU implementations.

1.5 Machine Vision Pipeline Diagram

The machine vision pipeline developed for this project integrates dataset preparation, model training, deployment, and evaluation into a cohesive workflow. Figure 1.1 provides an overview of the pipeline.

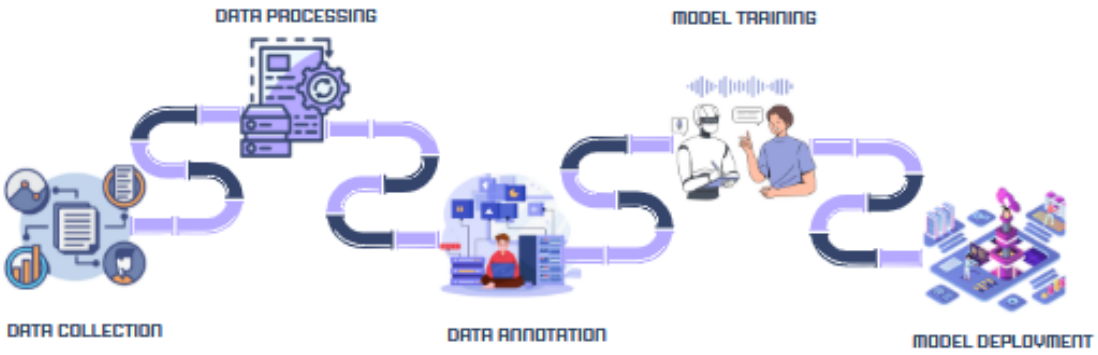


Figure 1.1: Machine Vision Pipeline for Steel Surface Defect Detection.

1.6 Report Outline

This report is structured as follows:

- **Chapter 2: Literature Review** explores existing research and solutions for defect detection using machine vision and FPGA technology.
- **Chapter 3: Data Processing** details the preparation, annotation, and augmentation of datasets used for training the machine vision pipeline.
- **Chapter 4: Model and Results** presents the YOLOv5n model, its architecture, and the experimental results, including key performance metrics.

- **Chapter 5: FPGA Deployment** discusses the quantization, compilation, and deployment of the trained model on FPGA hardware, highlighting its performance and efficiency.
- Appendices include supplementary materials such as datasets, technical documentation, and test cases.
- **Chapter 6: Results** A detailed outline of the results obtained by the steel surface defect detection system on the ultra96-v2 FPGA board, using inference outputs and graphs and matrices of important metrics like accuracy, precision, recall, mAP, FPS, F1 score, and power consumption.
- **Chapter 7: User Interface** The design and operation of the user interface created for the steel surface defect detecting system. Demonstrating how the system facilitates user-friendly interaction.
- **Chapter 8: Conclusion and Future Enhancements** The main accomplishments of the project are outlined. In addition to outlining possible future improvements, such as enhancing real-time processing, moving to more sophisticated FPGA platforms, and incorporating extra features.

Chapter 2

Literature Review

2.1 Introduction

The purpose of this chapter is to provide an overview of the existing research and studies related to machine vision and defect detection, particularly in steel manufacturing. It explores the use of machine learning models, FPGA implementations, and traditional approaches to understand their limitations and strengths.

2.2 Machine Vision in Steel Surface Inspection

This section reviews research on the application of machine vision systems for identifying and classifying defects on steel surfaces. Key topics include:

- Image processing techniques for defect detection.
- The role of deep learning models in improving accuracy and efficiency.
- Common challenges in steel surface inspection, such as variation in defect shapes and sizes.

2.2.1 EC-YOLO: Effectual Detection Model for Steel Strip Surface Defects Based on YOLO-V5

This paper proposes EC-YOLO, an improved version of YOLO-V5 based model, for detecting small and elongated defects on steel strips. To improve, they included an efficient channel attention bottleneck (EB) module, for better extraction of features of the strip, and Context Transformation Networks block for enhanced feature fusion and context modeling. They used publicly available datasets; GC10-DET, NEU-DET and SLD-DET, and achieved mean Average Precision (mAP) values of 71%, 83% and 87.5%,

respectively, on the improved model. The adapted methodology can mitigate the challenges of aliasing effects and loss of small defect details during downsampling.[4]

2.2.2 Lightweight Surface Defect Detection Algorithm Based on Improved YOLOv5

This paper proposes a lightweight surface defect detection algorithm based on improved YOLOv5 model for better accuracy and speed. They introduced a lightweight spatial attention module to extract effective information and designed a new feature fusion network, Amsf and added the FAM attention mechanism to enhance detection capability. Boundary loss errors are also reduced by replacing the GIoU loss function with the SIoU loss function. The experiment was carried out on the NEU-DET dataset, which gave an average detection accuracy of 81.97%.[28]

2.2.3 LCG-YOLO: A Real-Time Surface Defect Detection Method for Metal Components

In this paper, a customized YOLO-v5 model is used, which is LCG-YOLO for real-time defect detection for metal components. The LSandGlass (LSG) module is used to reduce information loss, and eliminates the low-resolution feature layer. The experiment was carried out on a self-made dataset, on which the mAP improved by 5.7% to 95.50%, and the FPS improved by 21 fps to 95 fps, compared to the original YOLO-v5.[29]

2.2.4 Metal Surface Defect Detection using Object Detection Models

This paper proposed the implementation of YOLOv5, YOLOv7, and Detectron2 model. They used their custom dataset for defect detection by merging and processing images from two datasets, NEU-DET, and GC10-DET. They achieve mAP scores of 77.4% on YOLOv5, 77.7% on YOLOv7, and 43.4% on Detectron2 model.[21]

2.2.5 An Enhanced YOLOv4 Model with Self-dependent Attentive Fusion and Component Randomized Mosaic Augmentation for Metal Surface Defect Detection

This paper proposed a modified YOLOv4 model for metal surface defect detection. The proposed method was experimented on self-developed MSDD dataset. The results show that their method can outperform several mature object detection algorithms; including

Faster R-CNN, YOLOv4, YOLOv5, and YOLOX by a large margin of 7.85%, 6.51%, 3.76%, and 3.57%, respectively. The modified model has the mAP score of 94.65%.[24]

2.3 FPGA Applications in Machine Learning

FPGA technology has gained prominence for its ability to accelerate machine learning models. This section discusses the research done on:

- Advantages of FPGA over traditional hardware like CPUs and GPUs for industrial applications.
- Existing implementations of FPGA-based machine learning pipelines for defect detection.
- Challenges in deploying deep learning models on FPGA platforms.

2.3.1 FPGA-Based Real-Time Object Detection and Classification System Using YOLO for Edge Computing

This paper proposed highly efficient Field Programmable Gate Array (FPGA) based real-time object detection and classification using YOLOv3 Tiny for edge computing. The proposed system evaluates its performance on Advanced Driving Assistance System (ADAS). A camera is connected to the *Kria KV260 FPGA development board* to detect and classify the traffic lights. They used Bosch Small Traffic Light Dataset (BSTLD) for model training and used *Xilinx Vitis AI* to quantify and compile the YOLO model. The system achieves 99% accuracy with only 3.5W power consumption.[1]

2.3.2 Design of a YOLO Model Accelerator Based on PYNQ Architecture

This paper proposed a YOLOv2 model accelerator architecture evaluated on *Xilinx-PYNQ z2 board* which is equipped with a Cortex A9 chip. The system adopts ARM+FPGA architecture to accelerate the inference. The weight size of the model is quantized from 194MB to 97MB, therefore reducing the consumption of the following resources.[25]

Table 2.3.2: Resource Utilization

Resource	Utilization	Available	Utilization %
LUT	26573	53200	50
FF	23794	106400	23
BRAM	88	140	63
DSP	151	220	69
MMCM	1	4	25

The designed hardware accelerator is simulated and synthesized in Vivado 2019.2 and then the software and hardware co-simulation is performed.

2.3.3 Design of Low-power YOLO Accelerator Based on FPGA

This paper proposes a low-power YOLOv4-Tiny algorithm accelerator based on FPGA. They customized the algorithm to reduce model's computational load, and to speed-up they maximized the use of pipelining, and loop unrolling operations. The improved algorithm increased the mean Average Precision (mAP) value by 1.9% on MS COCO dataset. Power consumption of this design is 3.415W. *Xilinx ZYNQ series XC7Z035 board* was used, equipped with a dual-core Cortex-A9 processor.[17]

2.3.4 Design and Implementation of the High-performance YOLO Accelerator based on Zynq FPGA

This paper implements the accelerators for YOLOv2, YOLOv2-tiny and Improving YOLO three models on ZYNQ FPGA platform. For high-performance and lightweight hardware acceleration system, software and hardware co design methodology is adopted. 16-bit dynamic quantization is used to reduce the size of the model. After quantization of YOLOv2, YOLOv2-Tiny and the improved YOLO model achieved the accuracy of 90.51%, 74.5%, and 81.5%, respectively.[27]

2.3.5 Resource-constrained FPGA implementation of YOLOv2

This paper proposed an FPGA-based YOLOv2 implementation for object detection tasks on low-resource CV-based IoT edge devices. Model is deployed on the *Xilinx Zynq UltraScale+ MPSoC ZU7EV board*, which provides the necessary resources for running the YOLOv2 algorithm efficiently. It utilizes Vivado HLS for hardware design. The final implementation achieves an mAP of 75%, a frame rate of 2.16 FPS, and power consumption of 4.8 W.[30]

2.3.6 A Power-Efficient Optimizing Framework FPGA Accelerator Based on Winograd for YOLO

This paper proposed a low-power and resource-constrained YOLOv2 object detection algorithm, deployed on an FPGA for IoT. Model is deployed on the *Xilinx PYNQ-z2 board*, utilizing Vivado HLS for hardware design. They used innovative dataflow and buffering techniques to optimize performance and resource utilization. The system achieves a mean Average Precision (mAP) of 78.25% with power consumption of 2.7 W.[2]

Table 2.1: Summary of research papers

Paper name	Dataset Name	Model Used	FPS	mAP@0.5
EC-YOLO: Effectual Detection Model for Steel Strip Surface Defects Based on YOLO-V5	GC10-DET & NEU-DET	EC-YOLO(Modified YOLOv5)	91.74	0.83
Lightweight Surface Defect Detection Algorithm Based on Improved YOLOv5	NEU-DET	LW-YOLO(Modified YOLOv5)	70.2	0.8197
LCG-YOLO: A Real-Time Surface Defect Detection Method for Metal Components	Self made	LCG-YOLO(Modified YOLOv5)	95	0.955
Metal Surface Defect Detection using Object Detection Models	GC10-DET + NEU-DET	YOLOv5 Small, YOLOv7, Detectron2	-	0.774, 0.777, 0.434
An Enhanced YOLOv4 Model with Self-dependent Attentive Fusion and Component Randomized Mosaic Augmentation for Metal Surface Defect Detection	self -made MSDD dataset	YOLOv4		0.946
FPGA-Based Real-Time Object Detection and Classification System Using YOLO for Edge Computing	BSTLD	YOLOv3 Tiny	15	0.99
Design of a YOLO Model Accelerator Based on PYNQ Architecture	COCO datatest	YOLOv2		
Design of Low-power YOLO Accelerator Based on FPGA	MS COCO Dataset	YOLOv4 Tiny	3.32	
Design and Implementation of the High-performance YOLO Accelerator based on Zynq FPGA	COCO test	YOLOv2, YOLOv2-Tiny, YOLOv2 improved	-	91.37, 90.51, 74.5, 81.5
Resource-constrained FPGA implementation of YOLOv2	PASCAL VOC 2007 and VOC 2012	YOLOv2		75
A Power-Efficient Optimizing Framework FPGA Accelerator Based on Winograd for YOLO	PASCAL VOC 2007 and VOC 2012	YOLOv2, Tiny YOLO, and SLIM YOLO	-	78.25,

2.4 Gap Analysis

The review concludes by identifying gaps in the existing literature, such as the lack of scalable, energy-efficient solutions for real-time defect detection. Existing models focus on dataset-specific improvements, achieving high accuracy (e.g., EC-YOLO: 87.5%,

LCG-YOLO: 95.5%) but lack generalizability to diverse defects, robust small-object detection, and a balance between accuracy and real-time speed.

Therefore, there is a need for an FPGA-based solution that integrates advanced YOLO models with efficient hardware utilization to achieve high accuracy, and scalability for diverse industrial applications.These gaps provide the motivation for this project's approach using an FPGA-based machine vision pipeline.

Chapter 3

Data Processing

3.1 Introduction

This chapter explains the methodology and techniques used to prepare the dataset for training and testing the machine vision pipeline for the detection of steel surface defects. The preparation of data was done on multiple datasets, as detailed in the upcoming subsection, 'dataset overview'. The data preparation focused on cleaning, transforming, and organizing raw data into a format that machine learning algorithm can understand, while ensuring compatibility with the YOLOv5 framework and effective management through version control tools.

The datasets were initially consisted of unannotated steel rod images which was processed and labeled using an online CV tool, Roboflow. Proper data preprocessing is critical to ensure accurate and efficient defect detection in machine vision tasks. The major steps for preprocessing of the data included; annotating the dataset, applying data augmentation techniques, and splitting the dataset into Train, Validation, and Test Sets and ensuring compatibility with the YOLOv5 framework and effective management through version control tools. These steps ensured that the datasets were well prepared for the machine vision pipeline.

3.2 Dataset Overview

The data processing workflow evolved through multiple stages as the project advanced. Initially, we experimented on publicly available dataset, NEU-DET dataset. To diversify the data, the NEU-DET dataset was combined with GC10-DET, another public dataset of steel surface anomalies. Finally, to cater the solution to real-world industrial challenges, we worked on an industry-specific dataset provided by Agriauto Industries Limited.

- **NEU-DET Dataset:** Initially, we experimented on publicly available dataset,

NEU-DET. The dataset has balanced image classification and consist of 1440 black and white images of 200x200 pixels in the training set and 360 in the validation set.

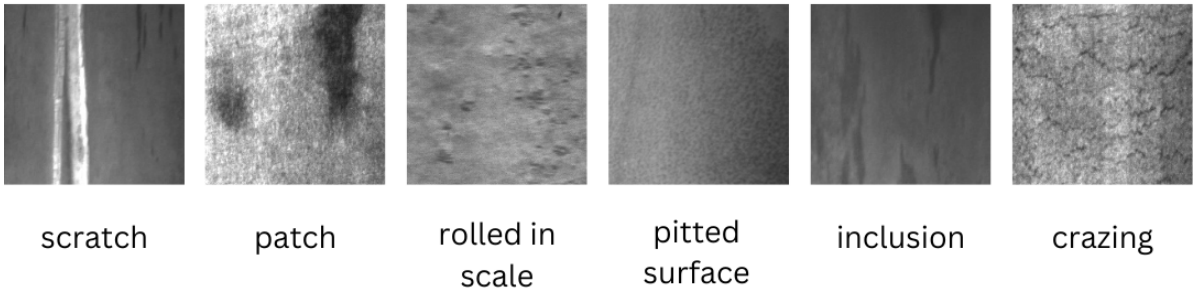


Figure 3.1: NEU-DET dataset defects

- **GC10-DET Dataset:** It is also a publicly available dataset for large-scale metallic surface defect detection. We combined the NEU-DET with GC10-DET dataset to expand the diversity and improve the accuracy.
- **Agriautos Dataset:** Agriauto Industries Limited provided us with the binary class dataset of unannotated images of steel rods ie; defected rods and undefected rods, featuring real-world defects on steel rods such as cracks, inclusions, and scratches. The dataset was manually annotated using Roboflow to create bounding boxes and class labels for supervised learning.

The dataset consists of steel rod images with various surface defects. These defects are labeled for supervised training of machine learning model. The primary defect categories includes:

- **Scratches:** Grooves or marks caused by mechanical abrasion on the surface.
- **Roughness** Uneven or coarse surface texture often caused by improper finishing.
- **Process pit:** A small hole or cavity.
- **Material pit** A tiny surface cavity caused by a limited loss of material.
- **Cracks** Sharp separations or fractures in the material due to stress or cooling defects.
- **Inclusions:** Compounds such as oxides and sulfides or impurities are embedded within the steel.

- **Dent** A surface indentation produced by pressure or impact from a hard object.

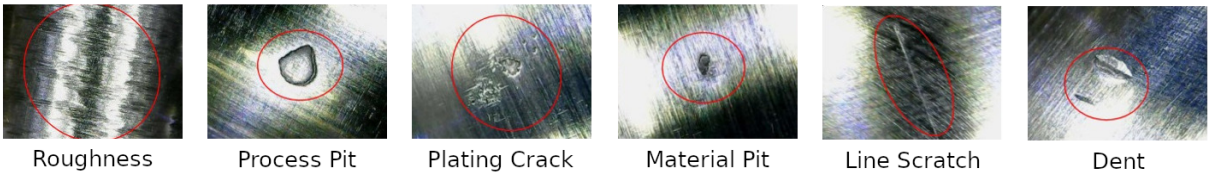


Figure 3.2: Defects on steel rods from Agriauto dataset

3.3 Data Preprocessing

For effective training of the machine vision pipeline, it is essential to make sure that the data is clean, well-organised, and ready for training the machine vision pipeline. This subsection delve into the key steps followed in preparing the dataset for defect detection.

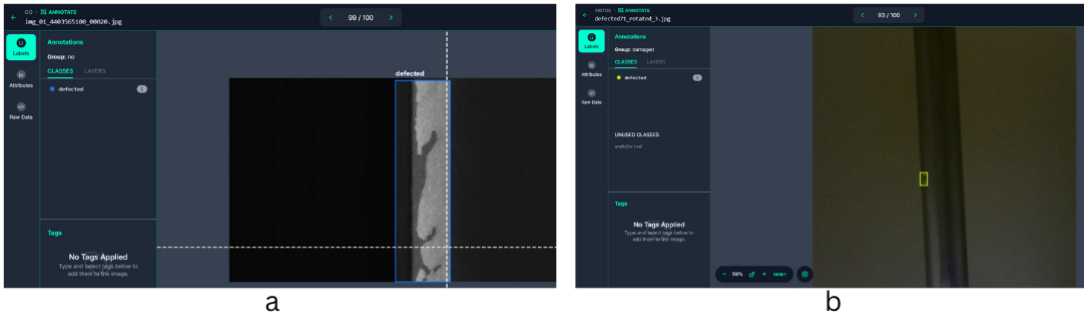
3.3.1 Dataset Preparation

The dataset preparation involved:

- **Data Acquisition:** We received our primary dataset from Agriautos Industries Limited, which consisted of unannotated images of steel rods with real-world defects. We then compared it with the publicly available datasets; NEU-DET and GC-10, to evaluate its diversity and improve the accuracy.
- **Data Inspection:** Ensured the quality of images by keeping the clear and free of noise images and discarding the low resolution and blurry images.
- **Data Standardization:** Resized all the images to a uniform resolution of 1080x1080 pixels to ensure compatibility with the machine learning model. All images were converted to a standard format, .png to ensure consistency and simplicity for the model.

3.3.2 Data Annotation

Data annotation is the process of identifying and labeling the content of images to ensure that the data is interpretable and usable by the computer vision model. As mentioned, the Agriauto dataset was unannotated, we manually annotated the dataset using an online CV tool, Roboflow. The defects in each image were labeled with bounding boxes to localize them and each defect was classified into categories such as cracks, inclusions, and scratches. Then the annotated data was exported to the YOLOv5 framework.



a b

Figure 3.3: Annotated images of NEU-DET(a) and Agriautos(b) Dataset using Roboflow

3.3.3 Data Augmentation

Data augmentation in machine learning is the technique to artificially increase the training set by creating modified copies of existing data. Different data augmentation techniques such as grey scale, rotation to range of degrees, zoom in zoom out were used.

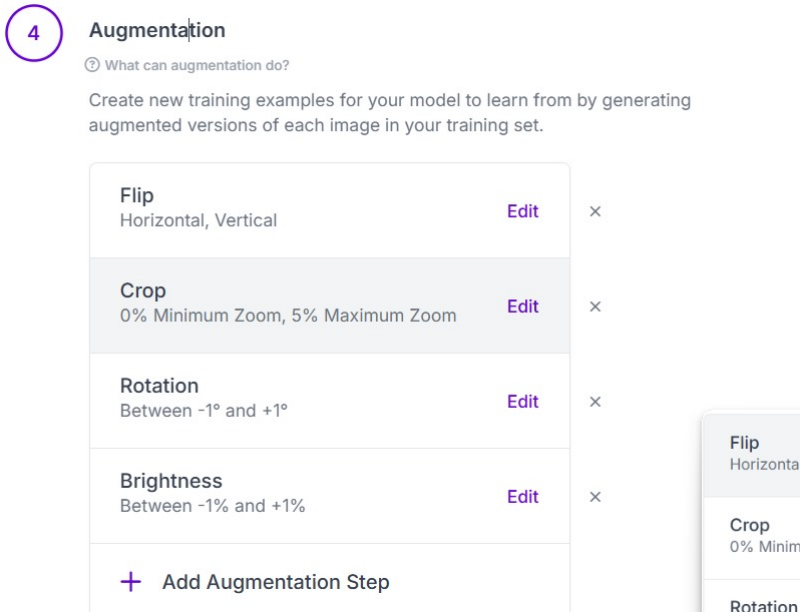


Figure 3.4: Dataset Augmentation Settings in Roboflow

1. **Augmentations Outputs per Training:**
   This refers to generating three augmented versions of each training example (image). By applying various transformations to the same base image, the dataset gains diversity, which helps improve the robustness and generalization of the model. The augmentations chosen are described below.

2. **90° Rotations**

These augmentations rotate the image in multiples of 90. It includes the following the variants:

- Clockwise (90°): Rotates the image 90° in the clockwise direction.
- Counter-Clockwise (-90°): Rotates the image 90° in the counter-clockwise direction.
- Upside Down (180°): Rotates the image 180°.

Purpose: Useful for handling object detection or classification tasks where objects in the dataset may appear at various orientations. For instance, this ensures the model can recognize objects irrespective of their rotational alignment.

3. **Crop**
   Crops a portion of the image based on a random zoom factor, keeping the object of interest within the cropped region. It has the following parameters:

   - Minimum Zoom (0%): No cropping is applied (the image remains as-is).
   - Maximum Zoom (10%): Crops up to 10% of the image dimensions, simulating a zoomed-in view of the scene.

   Purpose: Helps simulate changes in framing, mimicking variations in the distance of the camera to the object. This improves model robustness against slight spatial variations and partial object visibility.

4. **Rotation**
   Applies a small, random rotation to the image within a specific range of -10° to +10° (includes counter-clockwise and clockwise rotations).

   Purpose: Introduces variability to the object's orientation, addressing minor tilts or misalignment in the real-world data collection process.

5. **Cutout**
   Masks out random rectangular regions of the image to simulate occlusion.It includes the following parameters:

   - Number of Cutouts: 2 per image.
   - Size: Each cutout covers 10% of the image area.

   Purpose: Simulates partial occlusion of objects, ensuring that the model learns to detect objects even when parts of them are blocked. This is particularly useful in real-world scenarios like object detection in cluttered environments.

Integration in Dataset Synthesis

These augmentations collectively improve the training dataset by simulating real-world conditions like rotations, zoom levels, slight misalignments, and occlusions. By generating three augmented versions per base image, the dataset effectively expands in diversity, allowing the model to learn features that are invariant to these transformations.

3.3.4 Splitting Dataset

We split the dataset into three subsets using Roboflow:

- **Training Set:** 80% of the images were used to train the machine learning model.
- **Validation Set:** 10% of the images were used to fine-tune the hyperparameters and monitor the model and prevent overfitting.
- **Testing Set:** Remaining 10% was reserved to evaluate the final model's accuracy and performance.

3.3.5 Conversion and Exporting to YOLOv5-Compatible Format

To train the machine vision pipeline using YOLOv5, the dataset annotations and structure were converted into the YOLOv5-compatible format:

- **Annotation Conversion:** Annotations were exported as .txt files in the YOLOv5 format:

$$class\_id, x\_center, y\_center, width, height$$

.

- **Directory Structure Setup:** Images and labels were organized into folders as follows:
  - The images folder contains the raw images divided into train, val, and test subsets.
  - The labels folder contains the corresponding annotation .txt files, organized into the same subsets.

Chapter 4

Model and Results

4.1 Introduction

This chapter presents the core technical contributions of this project, focusing on the development and evaluation of a machine vision pipeline for steel surface defect detection. The chapter begins by outlining the dataset preparation process, which includes annotation, augmentation, and splitting strategies to ensure a well-balanced and representative dataset for training and validation.

The subsequent sections delve into the selection of the YOLOv5 architecture, detailing the research and experimentation conducted to identify the optimal variant for this application. Additionally, the chosen model is thoroughly examined in terms of its architectural components, with a specific focus on how its design supports real-time defect detection while maintaining low computational overhead.

The training process and optimization techniques are also discussed, including hyperparameter tuning and iterative experimentation to achieve the desired accuracy and robustness. Finally, the chapter concludes with a comprehensive analysis of the results, presenting key performance metrics, comparisons with alternative implementations, and visual examples to demonstrate the effectiveness of the proposed solution. This chapter provides a detailed account of the steps taken to ensure the model meets the performance and efficiency criteria required for industrial deployment.

4.2 Model Selection

We conducted extensive research on state-of-the-art object detection models to identify the most suitable architecture for steel surface defect detection. Several models were considered, including various versions of YOLO and RCCN-based architectures.

Key considerations included performance metrics, inference speed, compatibility with FPGA deployment methods, and compatibility with FPGA hardware.

Among the models evaluated, YOLOv11, YOLOv7, YOLOv3, and YOLOv5 stood out for their performance and resource utilization. However, after analyzing FPGA hardware constraints, deployment requirements and support documentation, we excluded YOLOv11 and YOLOv7.

4.2.1 Comparing YOLOv3 and YOLOv5

We extensively compared YOLOv3 and YOLOv5 to determine the better choice, the model parameters of YOLOv3 and YOLOv5 differ greatly, we needed a model that required the least amount of resources but also provided the best possible performance.

- **Architecure:** YOLOv3 uses a FPN neck with a cross-entropy loss function, while YOLOv5 uses a PANet neck with a binary-cross-entropy logit loss function. The head of both the models is the same.[7]
- **Speed:** YOLOv3 is faster than YOLOv5, with a processing time of 0.016 seconds per image compared to YOLOv5's 0.02 seconds per image.[9]
- **Accuracy:** YOLOv5 has better accuracy than YOLOv3 because it uses CSP-Darknet53, which is better at recognizing small and close objects than YOLOv3's Darknet-53.[19]
- **Resource Utilization:** YOLOv5 incorporates advancements in the in network design, the CSPDarknet reduces redundancy in feature map calculation leading to lesser reource utilization. Whereas YOLOv3 has a complex and more outdated architecture. YOLOv3 takes up 117MB while YOLOv5-small takes up 13.6MB.

After extensive research we concluded that YOLOv5 is the optimal choice for our purpose, it is better balance of speed, accuracy and resource utilization. Further experiments were conducted with different YOLOv5 variants, YOLOv5n, YOLOv5s, YOLOv5m, YOLOv5l. YOLOv5n emerged as the optimal choice due to its balance of high mean Average Precision (mAP), low memory consumption, and fast inference speed.

4.3 Model Architecture

The chosen YOLOv5n model features a Convolutional Neural Network (CNN) architecture designed for efficient real-time object detection. The key components of the model architecture include:

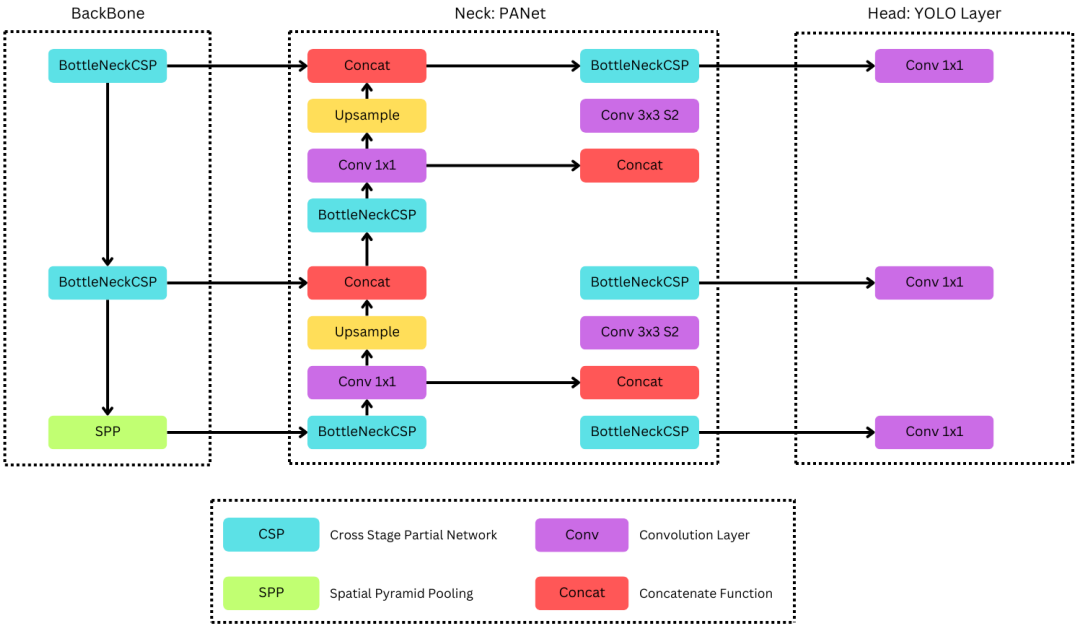


Figure 4.1: Yolov5 Model Architecture.

- **Backbone:** The role of the backbone is mainly to extract features from the original input images. It primarily comprises a number of convolutional layers required to identify edges, textures, and defect detection patterns.
- **Neck:** The neck gathers information from various scales to facilitate the network's capability to find out defects of diversified sizes. This is carried out using Feature Pyramid Networks (FPNs) and Path Aggregation Networks (PANs).
- **Head:** After that, the head outputs bounding boxes and class probabilities for all defects present in the input images.

4.3.1 CSP Backbone

YOLOv5 uses the Cross-Stage Partial module for feature extraction to enhance the backbone by addressing the issue of redundant gradient information in large convolutional neural network backbones. This architecture divides the feature maps into two main parts and then recombines them, reducing computational cost and model complexity without sacrificing performance.

- **Input:** The raw input image is resized to a standard size, such as 640x640 pixels, and normalized. Normalization typically involves subtracting the mean and dividing by the standard deviation of pixel values to stabilize the learning process.
- **Convolution:** The convolutional layers in the backbone apply filters to the input image to produce feature maps. These layers may use various kernel sizes, such

as 3×3 or 1×1, strides, and padding to maintain spatial dimensions.

- **Split:** The input feature map is split into two halves. One half undergoes a series of convolutional transformations, while the other bypasses this processing.
- **Transform:** The transformation path usually involves multiple convolutional layers, often using Batch Normalization and ReLU activations, to enhance the network's feature detection capabilities.
- **Merge:** Transformed and bypassed features are merged through concatenation, and another convolution is often applied to effectively combine these features.
- **Spatial Pyramid Pooling Fast(SPPF):** SPP layers aggregate features at multiple scales and are effective in maintaining spatial hierarchies among features. SPPF is a variation of SPP, specifically designed with three max pooling layers of the same kernel size (k = 5) arranged in parallel.[15]

4.3.2 PANet Neck

PANet (Path Aggregation Network) plays a critical role in enhancing feature representation by incorporating and refining features from various scales. This technique combines features from different layers through up-sampling, concatenation, and additional convolutional operations, enhancing feature hierarchy across the network and enabling more accurate object detection.

- **Upsampling:** Lower-resolution feature maps deeper in the network, which contain more abstract semantic information, are upsampled to match the resolution of higher-resolution, more detailed feature maps from earlier layers. Upsampling generally involves interpolation methods, such as nearest neighbor or bilinear interpolation.
- **Concatenation:** Upsampled feature maps are concatenated with corresponding high-resolution feature maps. This operation merges detailed textural information with high-level contextual information, creating a more robust set of features for the detection task.
- **BottleNeckCSP:** After the initial feature map concatenation, the aggregated feature maps pass through additional BottleNeckCSP blocks, similar to those in the backbone. These blocks help reduce redundancy, enrich features, and ensure efficient computation.[31]
- **Convolutional Layers:** The concatenated and processed feature maps are refined through additional convolutional layers. These layers include:

1x1 Convolutions: These reduce the depth of the feature maps, compacting information and reducing parameter counts. They act as feature selectors, transforming input features into a more useful form without altering spatial dimensions.

3x3 Convolutions: These broaden the receptive field and capture broader spatial patterns, which are essential for integrating local feature contexts across the aggregated feature maps.

4.3.3 Activation Function

YOLOv5 uses several activation functions throughout the network architecture to introduce non-linearity into the model for it to learn complex patterns from the data. The essence of these functions is quite critical in converting linear output from neurons to forms that can easily represent real-world data, like images in object detection tasks.

- **ReLU (Rectified Linear Unit):** ReLU is the most commonly used activation function in deep learning architectures due to its simplicity and efficiency.

  It is defined as:

  $$f(x) = \max(0, x) \tag{4.1}$$

  This is a neuron that is activated only when the input is positive; otherwise, it outputs zero. This property also helps in reducing the risk of a vanishing gradient problem in training. It can be mostly used in initial and middle convolutional layers of the network. This helps to accelerate training without causing significant degradation in the model's convergence ability.[5]

- **Leaky ReLU** Leaky ReLU is a variant of the ReLU function that allows a small, non-zero gradient when the unit is not active and the input is less than zero.

  It is defined as:

  $$f(x) = \begin{cases} x & \text{if } x \geq 0 \\ \alpha x & \text{if } x < 0 \end{cases} \tag{4.2}$$

  Here, αx is a small coefficient, for example, 0.1. This little slope in the negative region maintains information in the network, though at a reduced rate. Leaky ReLU is used especially in those layers that are deeper in the network where the chances of vanishing gradients are higher. It makes sure the process of gradient descent does not get stuck.

- **Mish:** Mish is a relatively new activation function that has been found to work well in deep learning models.

  It is defined as:

  $$f(x) = x \cdot \tanh\left(\ln\left(1 + e^{x}\right)\right) \tag{4.3}$$

Mish is a smooth, non-monotonic function that, while employed, provides a little slope even when the input may be negative,[5] helping the gradient propagate more smoothly through the architecture. Another variant of YOLOv5 employs Mish as a possible means for better training stability, ultimately boosting performance. This characteristic helps navigate the loss landscape more effectively.

- **SiLU/Swish (Sigmoid Linear Unit):** SiLU also known as Swish, is another activation function that combines aspects of sigmoid activation and linear transformation.

  It is defined as:

$$f(x) = x \cdot \sigma(x) \tag{4.4}$$

  where $\sigma(x)$ is the sigmoid function $\sigma(x) = \frac{1}{1+e^{-x}}$. SiLU is self-gating and allows the model to adaptively adjust the flow of information through the network[5]. SiLU has been adopted by YOLOv5 as the activation function of choice for various layers, most notably within the CSP blocks and the detection head. Several studies and empirical evidence indicate that SiLU can outperform ReLU in terms of identifying complex patterns and accelerating the convergence of the training process.

4.4 Model Optimization and Training

After selecting YOLOv5n as the target model, we customized its training process to suit our dataset. Key steps included:

- **Hyper-parameter Tuning:** Experiments were conducted with various hyperparameters, such as learning rate, batch size, learning rate, and epochs, to optimize the model's performance for our defect detection task. Table 4.4 shows the final hyperparameters used for training of our YOLOv5n model.

Table 4.4: Training Hyperparamters

Hyperparameter	Agri Autos	NEU-DET	NEU-DET+GC10-DET
Learning Rate	0.01	0.01	0.01
Epochs	10	160	160
Momentum	0.937	0.937	0.937
Wieght Decay	0.0005	0.0005	0.0005
Image Size	1280 x 1280	640 x 640	640 x 640
Batch Size	8	32	32

- **Model Training:** The model was trained for several hundred epochs on the training dataset. During this process, validation data was used to refine the hyperparameters further, ensuring minimal overfitting and optimal generalization.
- **Performance Metrics:** The final trained model achieved high accuracy and reliability in detecting steel surface defects, meeting the project's performance requirements.

4.5 Results and Analysis

4.5.1 Performance Metrics

The optimized model demonstrated the following performance metrics:

- **Precision:** Precision is defined as the proportion of true positives to all positive predictions. It shows how likely the model is for the expected bounding box to match the actual ground truth box.

$$\text{Precision} = \frac{(TP)}{(TP + FP)} \tag{4.5}$$

  where TP are the number of True Positive predictions and FP are the number of False Positive predictions.

- **Recall:** Recall is the fraction of true positives among all detections. It displays the probability of successfully detecting defects.

$$\text{Recall} = \frac{(TP)}{(TP + FN)} \tag{4.6}$$

  where FN are the number of False Negative predictions.

- **Mean Average Precision (mAP):** Achieved a high mAP across all defect classes.

$$\text{mAP} = \frac{1}{n} \sum_{k=1}^{n} AP_k \tag{4.7}$$

  where AP_k is the average precision of the particular class, n is the number of classes.[7]

4.5.2 Results

Table 4.5.2: Performance Metrics of Trained YOLOv5n Model

Performance Metrics	Agri Autos	NEU-DET	NEU-DET+GC10-DET
mean Average Precision (mAP50)	0.989	0.7873	0.803
Precision	0.98	0.761	0.827
Recall	0.988	0.722	0.714

4.5.3 Visualization of Results

Results are visualized through:

- Confusion matrices for defect classification accuracy.

  Figure 4.2 is the confusion matrix for the NEUDET+GC10DET dataset, it is nearly perfectly diagonal. The correctly predicted ”defected” samples show the normalized proportions of 0.98 while the correctly predicted ”non-defected” samples show the normalized proportions of 1.00. The off-diagonal cells show a false-negative rate of 0.02 i.e 2

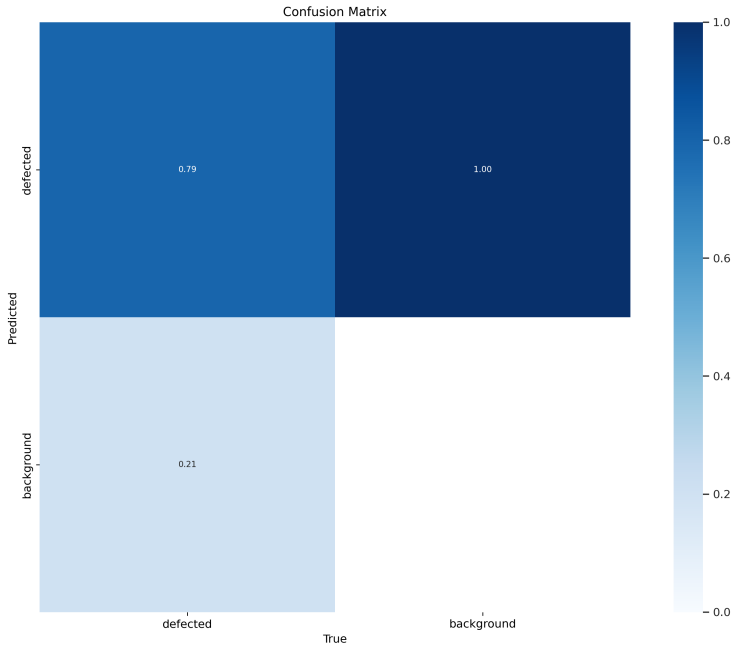


Figure 4.2: Confusion Matrix on NEUDET and GC10 Dataset

Figure 4.3 is the confusion matrix for the AgriAutos dataset, this matrix is less sharply diagonal, 0.79 ”defected samples” were correctly classified while 0.21 samples were misclassified as ”non-defected”. The model classifies ”non-defected”

samples with a perfect accuracy, indicating it does not produce false-positives. This performance shows the challanges in working with real-world datasets.

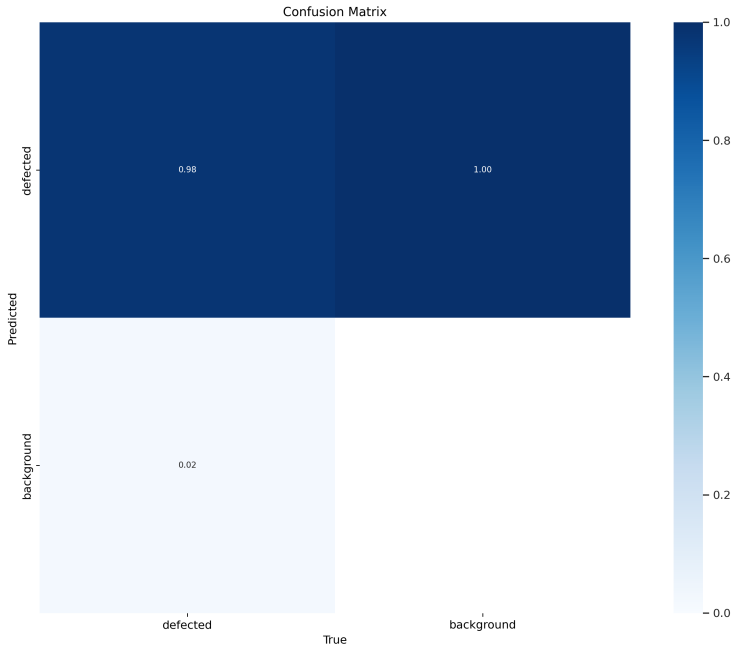


Figure 4.3: Confusion Matrix on AgriAutos Dataset

Figure 4.4 is the F1 curve for the NEUDET+GC10DET dataset, it shows the relationship between F1-score and the detection confidence threshold, which is the harmonic mean of precision and recall. F1-score initially increases to reach 0.74 with a threshold of 0.352 and then decreases as the threshold increases. At low thresholds, more predictions are made but more false-positives are introduced. At high thresholds, high confidence predictions are kept improving precision but missing true positives. The optimal threshold of 0.352 represents the best trade-off between precision and recall.

- F1 and precision curves

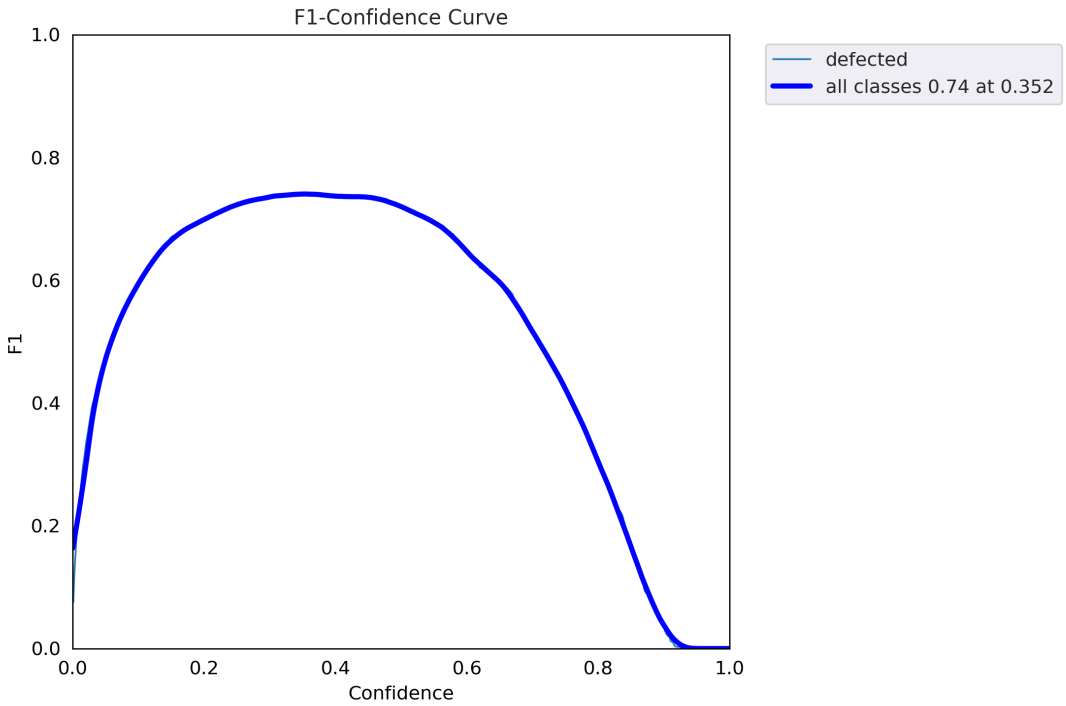


Figure 4.4: F1 cruve on NEUDET and GC10 Dataset

Figure 4.5 is the precision-recall cruve for the NEUDET+GC10DET dataset, this demonstrates the model achieves a maximum mAP@0.5 of 0.783. The curve begins with high precision for low recall and decreases as the recall increases which is a typical trade-off in object-detection. The higher area under the curve shows better model performance.

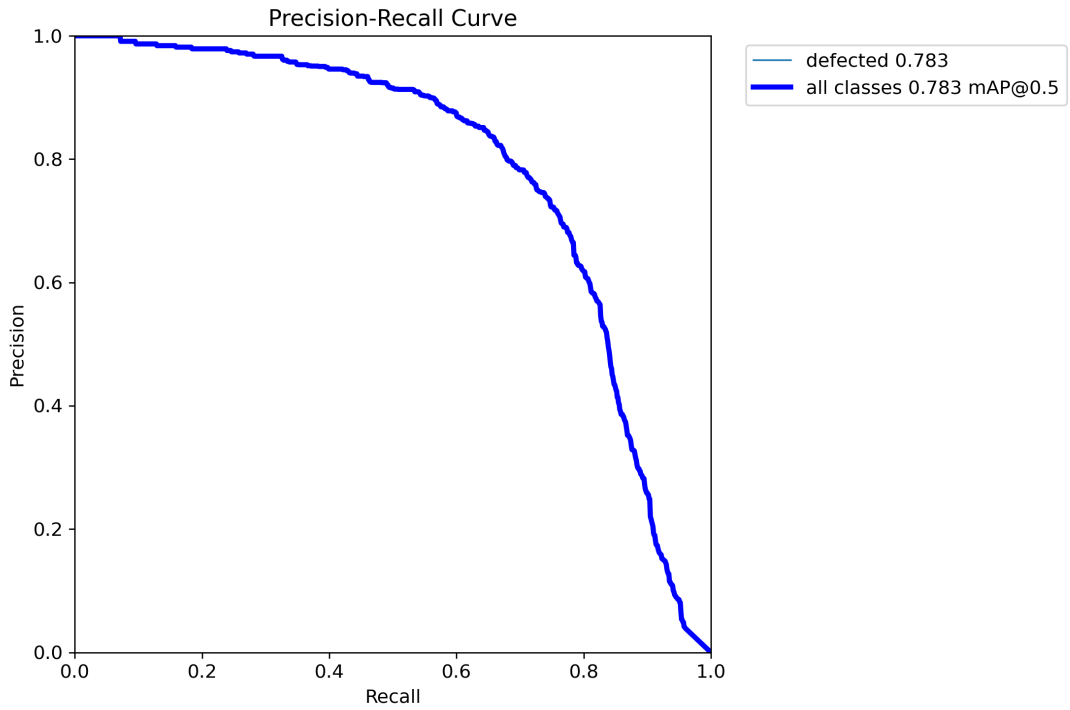


Figure 4.5: PR (Precision-Recall) curve on NEUDET and GC10 Dataset

Figure 4.6 is the F1 curve for the AgriAutos dataset, it shows the relationship between F1-score and the detection confidence threshold, which is the harmonic mean of precision and recall. F1-score peaks at 0.98 with a threshold of 0.688. At low thresholds, more predictions are made but more false-positives are introduced. At high thresholds, high confidence predictions are kept improving precision but missing true positives. The optimal threshold of 0.688 represents the best trade-off between precision and recall.

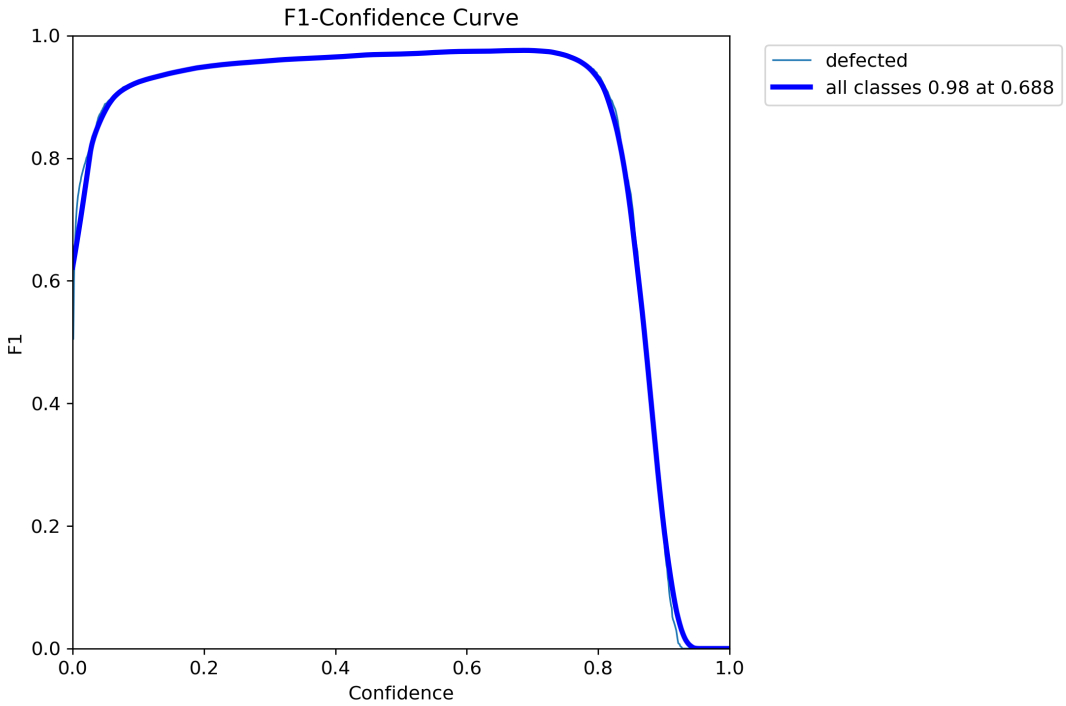


Figure 4.6: F1 scores AgriAuto Dataset

Figure 4.7 is the precision-recall cruve for the AgriAutos dataset, this demonstrates the model achieves a maximum mAP@0.5 of 0.977. The curve's shape shows the model can detect almost all defects with low false-positive rate, and the area under the curve confirms outstanding model performance.

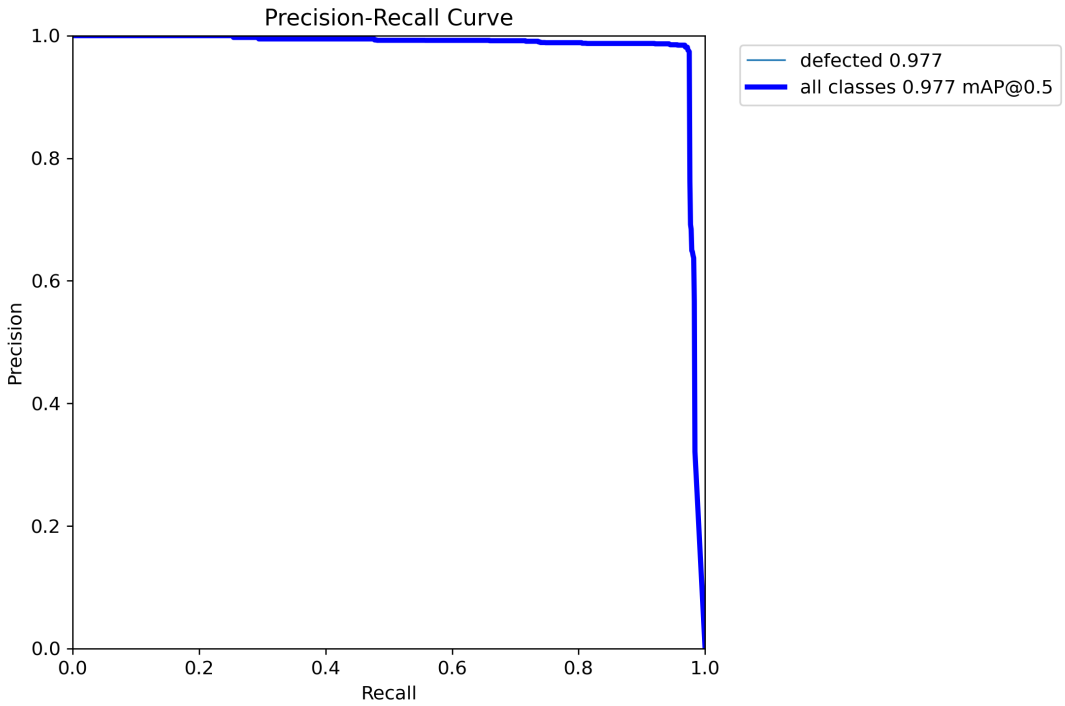


Figure 4.7: PR (Precision-Recall) on AgriAuto Dataset

- Sample defect images with predictions overlaid, showcasing the model's performance in real-world scenarios.

  Figure 4.8 shows sample inference results from the NEUDET+GC10DET datasets. Detected defects are highlighted with bounding boxes and their corresponding confidence scores. The confidence scores range from 0.39 to 0.78 indicating the model can detect subtle and prominent defects with different levels of certainty.

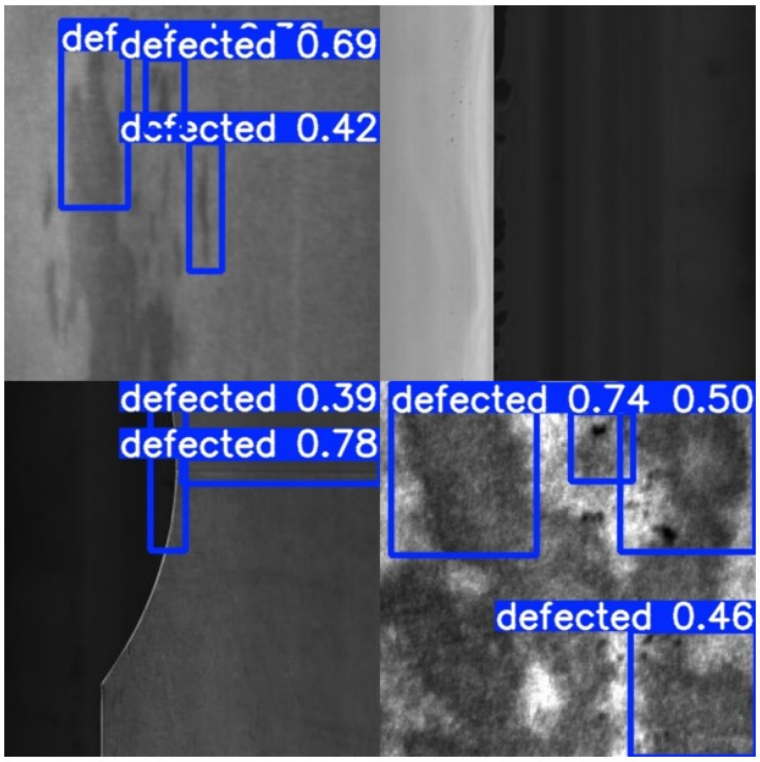


Figure 4.8: NEUDET+GC10 inference images on test set

Figure 4.9 shows the sample inference results from the AgriAutos dataset. Detected defects are highlighted with bounding boxes with high confidence indicating outstanding model performance.

Figure 4.9: AgriAuto test image inference

4.6 Discussion

The proposed solution demonstrates significant improvements in defect detection efficiency and scalability for industrial applications. Limitations, such as the dependency on annotated datasets and challenges in deploying larger models, are acknowledged. Future work includes exploring advanced FPGA architectures and expanding the dataset to cover additional defect types.

Chapter 5

FPGA Deployment

5.1 Introduction

This chapter focuses on the deployment of the YOLOv5 model using Vitis AI on the FGPA platform using Vitis-AI, a framework developed by Xilinx to accelerate AI inference on Xilinx hardware platforms, it consists of optimized IP cores, tools, libraries, models, and example designs. It is designed with high efficiency and ease of use in mind to unleash the full potential of AI acceleration on Xilinx SoCs. The Vitis AI workflow includes model preparation, inspection, quantization, compilation, and deployment on FPGA hardware.

5.2 FPGA Board

When picking an FPGA board, we had the following things to consider:

1. **Capability:** The FPGA board must have enough resources to be able to accelerate a deep learning model efficiently.
2. **Availability:** High end FPGA boards capable of hardware acceleration of models are expensive and not easy to procure.
3. **Development support:** Enough development tools and support documentation must be present for successful deployment.
4. **Peripheral Support:** The FPGA board must have support for peripheral devices for an interface to interact with the system.

After researching and checking availbility, we decided to pick the Avnet Ultra96-V2, which is an Arm-based, Zynq UltraScale+ MPSoC(Multiprocessor System on Chip)

FPGA development board based on the Linaro 96Boards Consumer Edition(CE) specification. Figure 5.1 shows the architecure overview of the ultra96-v2 MPSoC and Table 5.2 shows the available resources in the Programmable Logic(PL) of the FPGA chip.

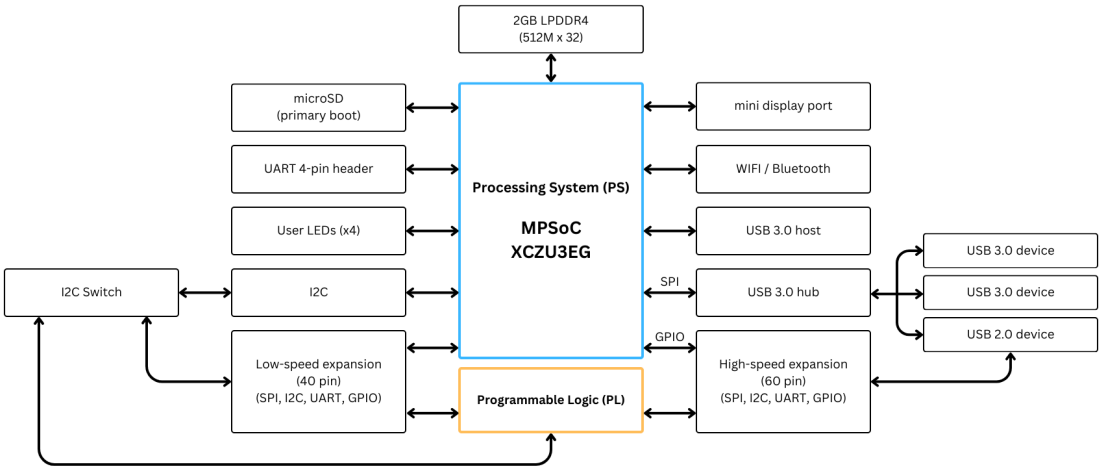


Figure 5.1: Ultra96-V2 Architecture.

Table 5.2: Resources Available on the Ultra96-v2 ZCU3EG FPGA

System Logic Cells	154,350
CLBs (Configureable Logic Blocks)	141,120
LUTs (Look Up Tables	70,560
Distributed RAM (Mb)	1.8
Block RAM Blocks	216
DSP (Digital Signal Processing) Slices	360
CMTs (Clock Management Tiles)	3

5.3 Vitis AI Deployment Workflow

Figure 5.2 illustrates the complete deployment workflow using Vitis AI, which includes the following steps:

1. **Model Preparation:** Preparing the model to ensure compatibility with the Vitis AI quantizer, including modifying model operators and verifying compliance with supported layers.

2. **Model Inspection:** Using the Vitis AI inspector to partition the model and identify which operators will run on the FPGA's Deep Processing Unit (DPU) or the CPU. This ensures maximum utilization of DPU resources.

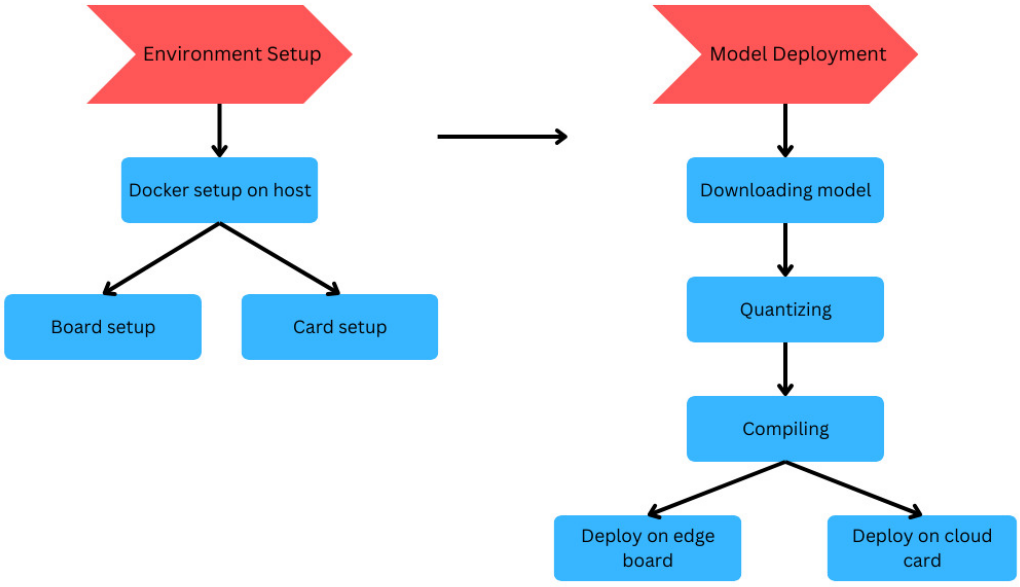


Figure 5.2: Vitis AI Deployment Workflow.

3. **Quantization:** Converting the model's floating-point weights and activations into fixed-point representations (e.g., INT8) to reduce computational complexity.

4. **Compilation:** Using the Vitis AI compiler to convert the quantized model into a deployable DPU-compatible format.

5. **Deployment:** Integrating the compiled model into the FPGA system and testing it for performance metrics such as speed, accuracy, and energy efficiency.

5.4 Model Inspection

Before quantization, the model is inspected using the Vitis AI inspector tool. This step ensures that the model is compatible with FPGA hardware by analyzing its operators and partitioning them into components that will run on either the DPU or CPU. The inspector outputs partition information, highlighting any unsupported operators and suggesting modifications to optimize the model for DPU execution. The model to be deployed should meet the following requirements:

- The model to be quantized should include forward method only. All other functions should be moved outside or move to a derived class. These functions usually work as pre-processing and post-processing. If they are not moved outside, the API removes them in the quantized module, which causes unexpected behavior when forwarding the quantized module.

- The float model should pass the jit trace test. Where the model is loaded using a pytorch method, torch.jit.trace, if the float model can be successfully loaded using this method then it can be quantized and the quantized model can also be loaded using this method.
- The float model should use only those operators which are supported by the Vitis AI quantizer.

5.5 Model Preparation

The trained YOLOv5n float model was inspected using the Vitis AI inspector upon which we discovered the folllowing issues:

- By default the activation function used by YOLOv5 is SiLU which is not supported by the Vitis AI quantizer, only Relu and LeakyRelu are supported. Vanishing gradient is the main problem with ReLU, LeakyReLU is an extension of ReLU which has been used in many deep learning models with promising performance[5]. Vitis AI supports LeakyRelu activation function only with a negative slope of 25/256 or 0.1015625.

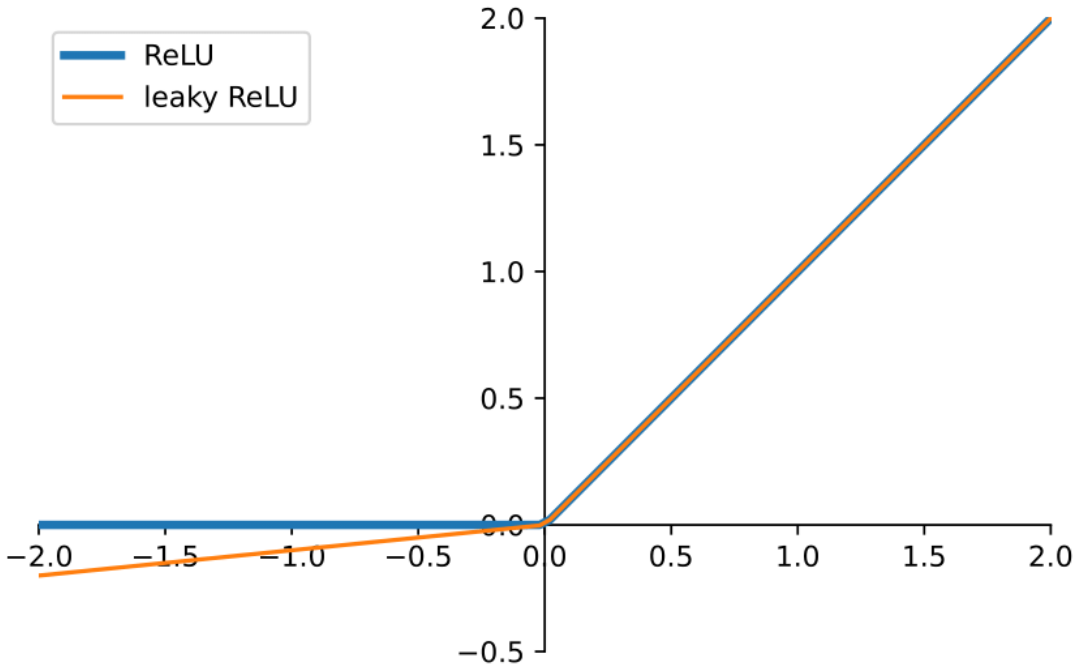


Figure 5.3: Relu vs LeakyRelu.

- In the last layer of the detection head of the float model there are permute and view functions used which are legacy YOLOv3 features and are not supported by Vitis

AI, these operators will prevent the model from being quantized and deployed. We removed the last layer of the detection head and then re-implemented the same in the post processing section.

The float model was then re-trained to implement the changes made in it's architecture, the model was then passed through the jit trace test and it passed.

5.6 Quantization

Quantization is a pivotal process in optimizing neural networks for deployment on FPGA hardware. It involves reducing the precision of the model's weights and activations from high-precision floating-point representations (e.g., 32-bit) to lower-precision fixed-point or integer formats (e.g., 8-bit). It can be applied to either weights or weights and activation. The convolution operation requires many addition and multiplication operations, the primary resources consumed in these operations are DSP slices and LUTs[18], The specific resource consumption is outlined in Table 5.6

Table 5.6: Resources Utilized in Floating Point Operations

Operation Types	DSP Slices	LUTs
Addition (32-bit floating point)	2	214
Multiplication (32-bit floating point)	3	135
Addition (16-bit floating point	0	47
Multiplication (16-bit floating point)	1	101

There are three types of benefits using the quantization technique:

- Reduction in computational time, computational complexity and required memory bandwidth.
- A balance is maintained between computational complexity and prediction accuracy.
- Quantization transforms the algorithms floating-point data operations to fixed-point multiply-add operations. Making it more suitable for FPGA platform computation.[18]

Given the above reasoning the 8-bit integer representation is the most used option, since the computations can be performed by the ALU without an FPU. it is also the minimum unit register format inside the CPU and the RAM since it is byte addressed. An even smaller representation would require either custom hardware support and can result in severe performance drops.[16]

5.6.1 Vitis AI Quantizer Workflow

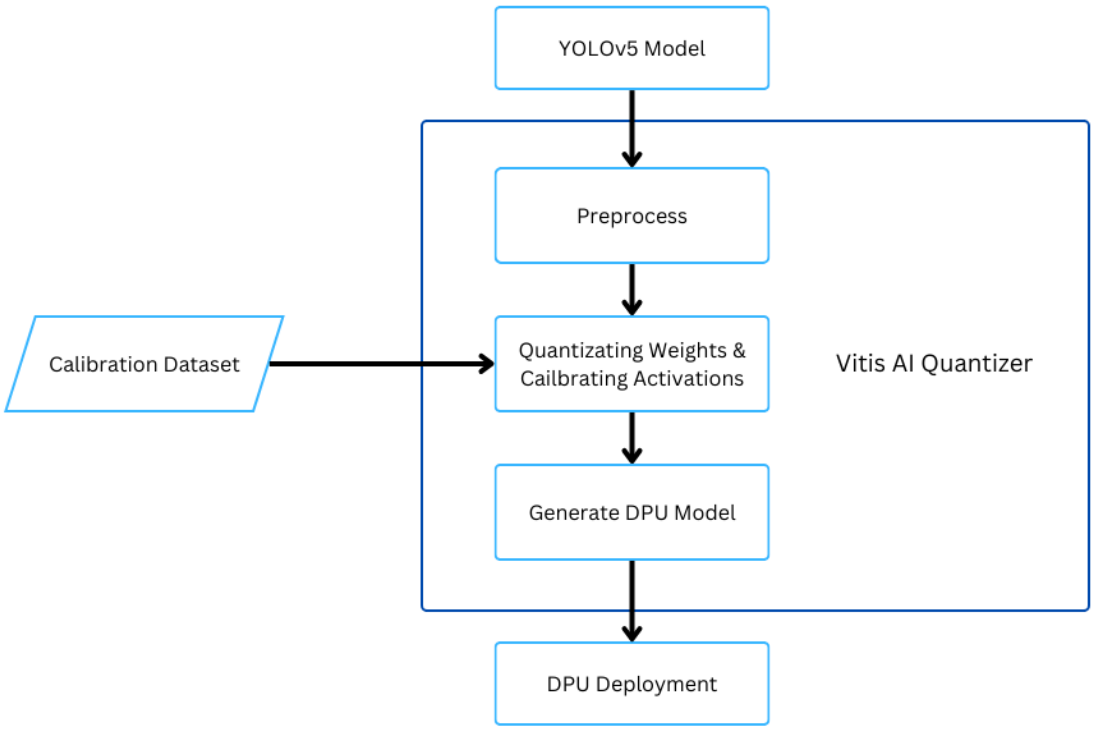


Figure 5.4: Quantizer Workflow in Vitis AI.

Quantization is performed using the PyTorch version of the Vitis AI quantizer, *pytorch_nndct* packaged with NNDCT(Neural Network Deep Compression Toolkit) containing all tools necessary for quantization of a float model in the PyTorch format. It streamlines the quantization process through a structured workflow:

1. **Preprocessing:** This initial step involves folding batch normalization layers into preceding convolutional layers and eliminating nodes that are redundant during inference. Such optimizations simplify the model and enhance its compatibility with fixed-point arithmetic.

2. **Calibration:** Utilizing a calibration dataset, typically comprising 100 to 1000 unlabeled images, the quantizer assesses the distribution of activations. This analysis determines appropriate scaling factors, ensuring that the reduced-precision representations maintain the dynamic range necessary for accurate inference. A quantizable module is generated.[26]

3. **Generate DPU Model:** The quantizer performs quantization and exports the quantized model in a format compatible with the FPGA's Deep Processing Unit (DPU), as `.xmodel` which will be later compiled and deployed on the FPGA board

along with `.pt(PyTorch)` and `.onnx(TensorFlow)` for evaluating the quantized model before deploying it in a production environment.

5.6.2 Quantization Techniques

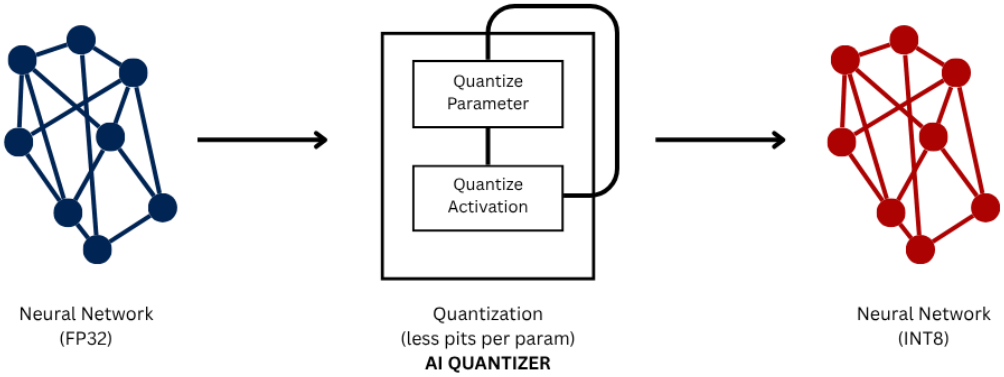


Figure 5.5: Quantization with Vitis AI.

The decimal mathematical expression for fixed-point numbers is illustrated in Equation (4), where *bw* is the bit width of the fixed-point number, *fl* is the length of the fractional part, B_{i} represents the quantization precision, B_{i} is between 0 and the quantization base, and the most significant bit denotes the sign.[18]

$$V_{\text{int}} = \sum_{i=0}^{bw-1} 2^{-f1} \times B_i \times 2^i \tag{5.0}$$

Vitis AI supports two primary quantization methodologies:

- **Post-Training Quantization (PTQ):** This approach quantizes a pre-trained floating-point model without additional training. It requires a calibration dataset to determine quantization parameters. While PTQ is efficient and straightforward, certain network architectures may experience notable accuracy loss. In such scenarios, advanced calibration techniques like fast finetuning can be employed to enhance performance.
- **Quantization-Aware Training (QAT):** Integrating quantization into the training process, QAT simulates the effects of quantization during model training. This method enables the model to adapt to quantization-induced constraints, often resulting in higher accuracy compared to PTQ.[16] However, QAT demands access to the original training dataset and additional training epochs.

5.6.3 Quantization Process

In this project PTQ(Post Training Quantization) was used with Vitis-AI, mapping floating point values to integer values using linear mapping through the range of tensor values extracted in the calibration phase.

1. **Calibration:** A small set of images from the original dataset are passed through the trained FP32 model the minimum

   $$x_{\text{min}}$$

   and maximum

   $$x_{\text{max}}$$

   for the weights and activations of each layer are extracted.[10]

2. **Scale:** A scale values is then determined, it defines how a range of floating-point values are mapped to integer values (such as INT8, ranges from -128 to 127)[8]. Represents the size of a single integer ”step” in the floating-point domain. Suppose a tensor has the following FP32 values:

   $$x = [-0.85,\ 0.67,\ -0.30,\ 0.12,\ 0.55,\ -0.92,\ 0.0,\ 0.44]$$

   Calculate the scale:

   $$x_{\text{min}} = -0.92 \tag{5.1}$$

   $$x_{\text{max}} = 0.67 \tag{5.2}$$

   $$\text{scale} = \frac{\max(|-0.92|,\ |0.67|)}{127} = \frac{0.92}{127} \approx 0.00724 \tag{5.3}$$

3. **Zero Point:** The exact representation of the 32-bit floating-point value of zero in 8-bit integer. This value is zero in symmetric quantization.[8]

4. **Quantization Formula:** Quantizing the 32-bit floating-point tensor value to 8-bit integer is done as follows:

   - **Symmetric Quantization (commonly used for weights):**

     $$x_{\text{int8}} = \text{round}\left(\frac{x_{\text{fp32}}}{\text{scale}}\right) \tag{5.4}$$

     where

     $$\text{scale} = \frac{\max\left(|x_{\text{min}}|, |x_{\text{max}}|\right)}{127} \tag{5.5}$$

     x_{min} and x_{max} are the minimum and maximum FP32 values in the tensor.[10]

- **Asymmetric Quantization (possible for activations):**

$$x_{\text{int8}} = \text{round}\left(\frac{x_{\text{fp32}} - x_{\text{min}}}{\text{scale}}\right) + \text{int8}_{\text{min}} \tag{5.6}$$

where

$$\text{scale} = \frac{x_{\text{max}} - x_{\text{min}}}{\text{int8}_{\text{max}} - \text{int8}_{\text{min}}} \tag{5.7}$$

- **Example:** Quantizing $x_{\text{fp32}} = 0.67$:

$$x_{\text{int8}} = \text{round}\left(\frac{0.67}{0.00724}\right) \tag{5.8}$$

$$= \text{round}(92.6) \tag{5.9}$$

$$= 93 \tag{5.10}$$

5. **Dequantization:** To recover the original approximate floating-point value of the quantized tensor for inference.

$$x_{\text{fp32}} \approx (x_{\text{int8}} - z) \times \text{scale} \tag{5.11}$$

where z is the zero-point (often zero for symmetric quantization).

5.7 Compiling

5.7.1 Overview

The quantized model is compiled to convert it into a hardware executable format for inference on the hardware. This is necessary for capitalizing on the processing power of the Deep Processing Unit(DPU) of the Ultra96-V2 board. This is done using the Vitis-AI compiler, ensuring the compiled model's architecture is compatible on the FPGA board.

5.7.2 Methodology

- **Input Model:** the model is quantized and exported in the .xmodel format, with INT8 weights and activations. The compiler requires an arch.json with the fingerprint of the DPU on the Ultra96-V2 FPGA board.[12]
- **Model parsing and Graph Analysis:** the model is parsed, analyzing the computational graph, identifying all the layers, operations and connections. The model is checked for supported layer types and compatibility of the operators with the FPGA board.

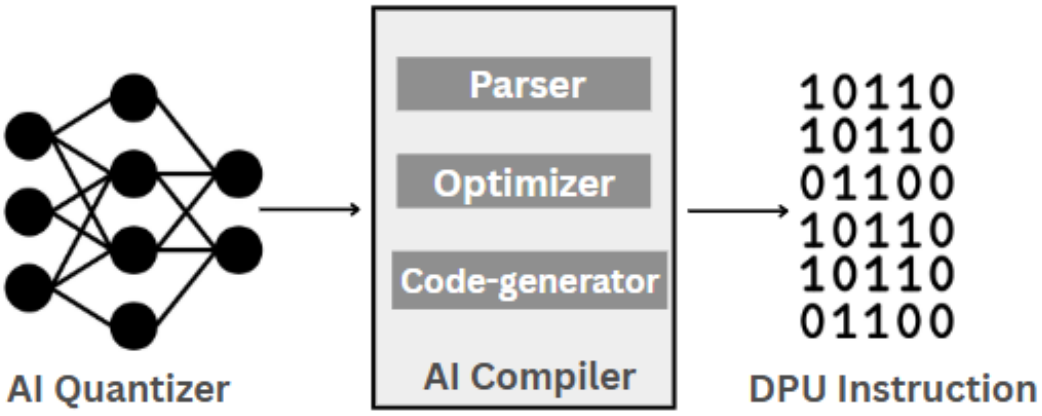


Figure 5.6: Compiling with Vitis AI.

- **Graph Optimization:** the compiler applies optimizations to improve the model's efficiency:
  1. **Operator Fusion:** merge adjacent layers, convolution, batch normalization, and activation functions into single hardware functions to reduce resource usage and latency.[11]
  2. **Constant Folding:** computes constant expression at compile time, reducing computational load.
  3. **Pruning and Reordering:** removes redundant operations and optimize the order of computations.[22]
- **Subgraph Partitioning:** the DPU computational graphs are divided into two:
  1. **DPU Subgraphs:** segments of the network that will execute on the FPGA DPU.
  2. **CPU Subgraphs:** segments of the network that must run on the ARM CPU.[14]
- **Instruction Generation:** for each DPU subgraph the compiler translates the neural network computations into low-level DPU executable instructions[23]. This includes:
  1. Allocating memory buffers for input, output and intermediate tensors.
  2. Arrange the order of operations for processing
  3. Optimizing memory access to fully utilize the onboard memory instead of external DDR bandwidth.
- **Output:** an FPGA ready model in the .xmodel format is produced with DPU-specific instructions and layouts.[6]

5.8 DPU-PYNQ

5.8.1 Overview

DPU-PYNQ is an open-source framework developed by Xilinx, enabling users to deploy deep learning models on various FPGA boards. It is a Linux-based OS for the FPGA boards, that utilizes the DPU for inference implemented in the programmable logic (PL) of the FPGA board[13]

Framework Goals

The DPU-PYNQ is engineered to connect high-performance FPGAs with software environments.

1. **Simplified AI Deployment on FPGAs:** It provides Jupyter Notebooks and Python APIs for a user-friendly interface to facilitate non-hardware experts to run machine learning/deep learning inference on FPGA boards.

2. **Hardware-Accelerated Inference:** Integrated Deep Processing Unit (DPU) overlay for offloading compute-intensive operations (e.g., convolutions) from the CPU to FPGA fabric, exponentially improving inference performance.

3. **Support Pre-trained Model Integration:** Users can compile, deploy, and quantize models from popular machine learning frameworks such as TensorFlow, Keras, and PyTorch.

4. **Modular and Reusable Overlays** DPU configurations can be packaged with unique hardware accelerators or interfaces using PYNQ overlays, which facilitates their integration into larger systems or domain-specific applications.

5. **Promote Rapid Prototyping and Research:** Enable quick testing of AI models on edge devices like the Ultra96-V2, helping researchers and developers prototype embedded AI solutions without low-level HDL programming.

6. **Maintain Low Power Edge AI:** Optimize for low-power, real-time inference use-cases typical of edge AI scenarios, such as object detection, image classification, and speech recognition on constrained hardware.

Key Components

The DPU-PYNQ framework is built upon several modular components that work together to allow easy and efficient deployment of deep learning models on FPGA platforms. The key components include:

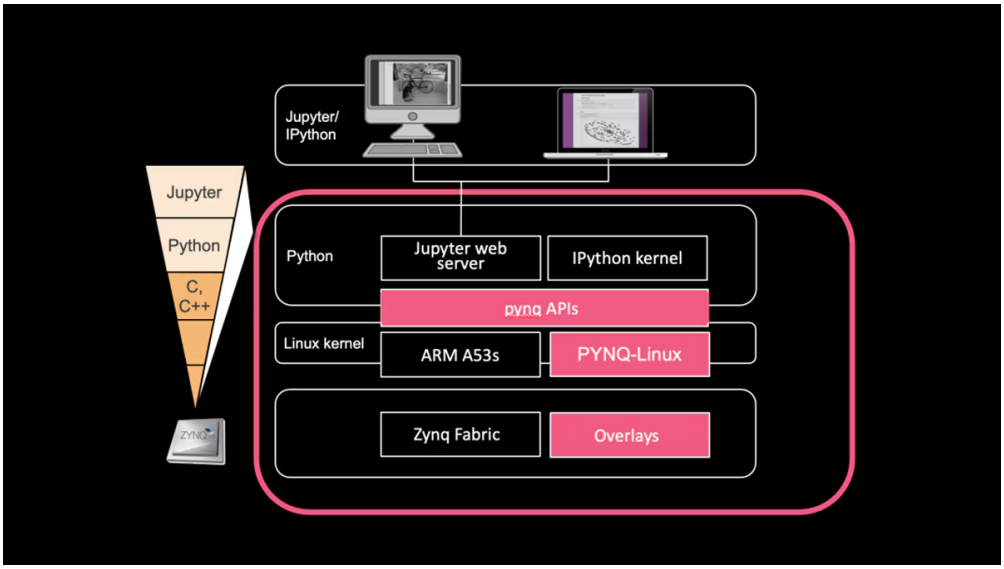


Figure 5.7: PYNQ DPU Architecture

1. **Bridging Python and Zynq for Enhanced Productivity:** PYNQ is a free and open-source framework developed by Xilinx. It allows users to use the programmable logic (PL) and processing system of Zynq SoCs through the use of Python. It features

   (a) a lightweight Linux distribution (PYNQ Linux),

   (b) operating on ARM processor cores of the Zynq device.

   (c) through Python libraries, Jupyter Notebooks, and remote SSH access, PYNQ enables simple interaction with FPGA logic.

2. **DPU (Deep Processing Unit):** The DPU is a pre-designed, highly-optimized hardware IP core provided by Xilinx for accelerating AI inference tasks. The DPU, running on the FPGA's programmable logic (PL), supports common CNN operators like convolution, ReLU, and pooling. It communicates with the ARM processor via AXI interfaces and executes quantized deep learning models generated by the Vitis AI Compiler.[3]

3. **Overlay:**

   - An overlay is a bit-stream file bundled with Python drivers, metadata, and hardware configuration.
   - It defines how the programmable logic (PL) on the FPGA is configured.
   - In DPU-PYNQ, the overlay includes:
     - The DPU architecture
     - Peripheral IP blocks including DMA, and memory controllers.

- Overlays are dynamically loaded in Python using the ‘Overlay‘ class from the ‘pynq‘ package.
- Once loaded, users can interact with hardware from high-level Python code.

4. **Python API and Runtime:**
    - DPU-PYNQ offers high-level Python APIs for DPU interaction within Python scripts or Jupyter Notebooks.[20]
    - These APIs abstract away low-level hardware interactions and simplify AI inference on FPGAs.
    - Key capabilities include:
        - Loading the DPU overlay (`Overlay` class)
        - Creating and managing a DPU runner (`DpuRunner`)
        - Allocating and transferring tensors between DRAM and DPU
        - Performing pre-processing (e.g., image resizing, normalization)
        - Running inference (synchronously or asynchronously)
        - Post-processing model output for visualization or further computation
    - Enables users without HDL or FPGA expertise to use powerful hardware acceleration.

Key Features

- **Hardware Acceleration:** Computationally intensive operations of the network are executed by the DPU in the FPGA fabric.
- **Python APIs:** Python interfaces for model loading, input processing, inference and result handling.
- **Seamless Vitis-AI Integration:** DPU-PYNQ can run models quantized and compiled by Vitis-AI.
- **Flexible Integration**

5.8.2 Hardware and Software Stack

The DPU-PYNQ framework leverages an integrated hardware-software stack, enabling efficient AI inference on Ultra96v2 (and other FPGA-based platforms). The key components of this stack include:

FPGA Board Architecture:

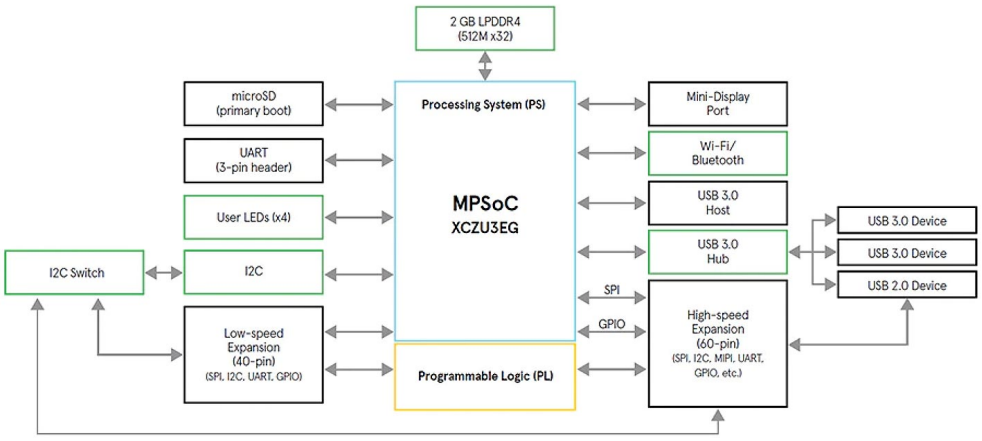


Figure 5.8: ultra96v2 board architecture diagram

1. ARM Cortex-A53 Processing System (PS) and programmable logic (PL) are combined on a single chip in Zynq UltraScale+ MPSoC
2. The PS runs the operating system and application logic, while the PL hosts the DPU and custom accelerators.
3. It includes high-speed interconnects (AXI) for communication between PS and PL.
4. Common boards include the **Ultra96v2**, and ZCU102.

Linux OS (PYNQ Linux):

1. PYNQ Linux is a lightweight Ubuntu-based distribution tailored for Xilinx boards.
2. It runs on the ARM cores and provides a Jupyter Notebook server, Python support, and pre-installed drivers.
3. It simplifies development by abstracting low-level hardware operations through Python APIs.
4. Users can interact with overlays, peripherals, and DPU using familiar scripting environments.

DPU Hardware IP Block:

- The Deep Processing Unit (DPU) is a configurable IP core for accelerating deep learning inference.
- It supports quantized CNN operators such as convolution, ReLU, pooling, and fully connected layers.
- Designed to execute models compiled by the Vitis AI Compiler, optimized for low latency and high throughput.
- The DPU connects to memory and DMA engines using AXI interfaces in the PL.

DPU Runtime and Drivers:

- The DPU runtime consists of user-space libraries and kernel modules that manage execution on the DPU.
- It includes the `libdnndpu` runtime library and associated drivers to handle memory allocation, task queuing, and inference execution.
- Python bindings (via PYNQ) allow easy access to these runtime services through the `DpuRunner` class.
- Ensures smooth coordination between software applications and the DPU IP core in hardware.

5.8.3 Software Flow for Model Deployment

The DPU-PYNQ deployment process consists of several key stages, each building on the previous. These stages transform a trained model into an optimized, hardware-executable form deployable on the FPGA.

Model Preparation (Training and Exporting)

Initially, a deep learning model is trained with PyTorch or Tensorflow framework. Following training, it is exported in a format that is compatible, like:

- TensorFlow: `.pb`
- PyTorch: `.pt` or converted to `.onnx`

Quantization using Vitis AI

The exported model is quantized to 8-bit fixed-point format using the Vitis AI Quantizer. This process:

- Reduces memory and computational cost
- Retains acceptable accuracy via calibration data
- Outputs a quantized model ready for hardware compilation

Compilation for DPU Target (VAI Compiler)

The quantized model is compiled using the Vitis AI Compiler (`vai_c_pytorch`) which:

- Maps neural network layers to DPU instructions
- Generates files like `model.json`, `model.bin`, and `meta.json`
- Targets the architecture of a specific DPU (e.g., B4096, B3136)

Bit-stream and Overlay Generation

The DPU hardware architecture is integrated into a Vivado design, which includes:

- DPU IP core
- AXI interfaces, DMA engines, and memory controllers

The design is packaged into a bit-stream and Python-compatible overlay (with drivers and metadata) for use with PYNQ.

Deployment on PYNQ Board

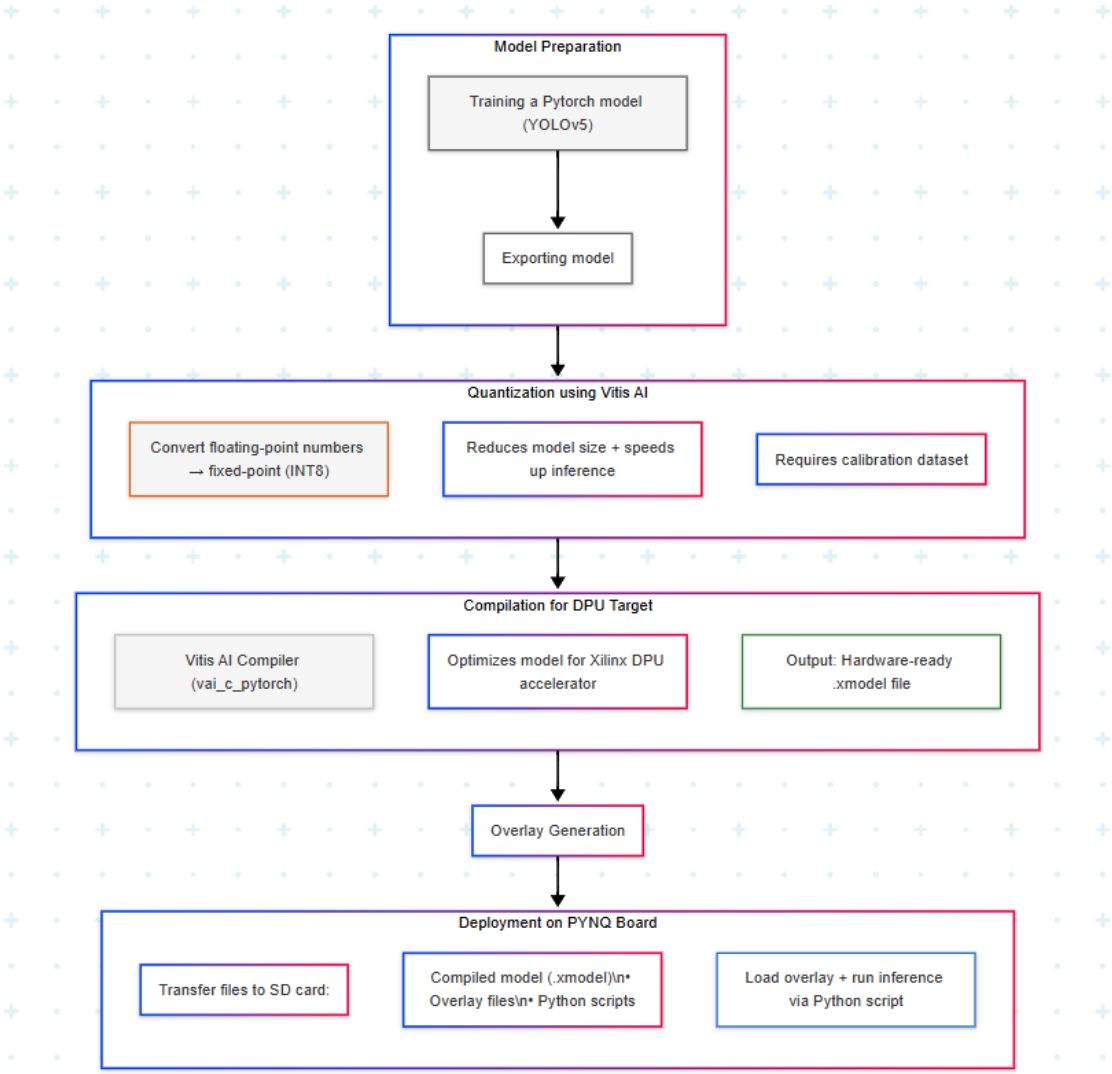


Figure 5.9: Software Flow

The final deployment process involves:

- Transferring the overlay and compiled model files to the board
- Loading the overlay using the `Overlay` class in Python
- Initializing the `DpuRunner` for managing inference
- Sending input data to the DPU, executing inference, and reading results

This pipeline enables complete model deployment and testing directly on FPGA with minimal low-level interaction.

5.8.4 Architecture of the DPU

The Deep Processing Unit (DPU) is a Xilinx-designed, highly optimized hardware IP block for efficient, parallel, and low-latency deep neural network inference on FPGA platforms, specifically engineered to minimize memory bottlenecks.

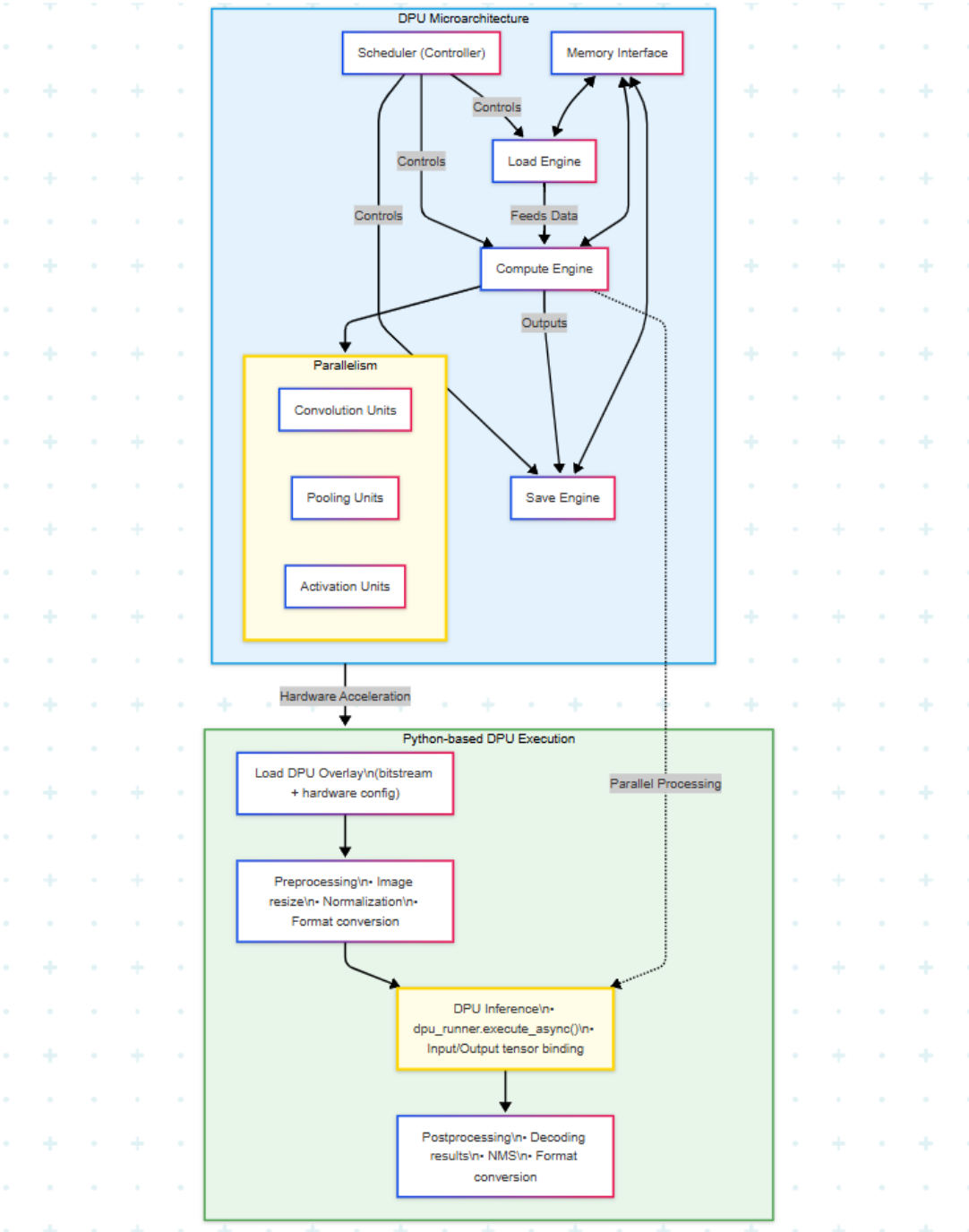


Figure 5.10: DPU Micro-architecture

The DPU incorporates specialized, pipelined hardware modules to support a range operations,ns including convolution, activation (ReLU, LeakyReLU), pooling, batch

normalization, and element-wise arithmetic.

The DPU IP is customizable — users can choose the number of compute units (CUs), DSP usage, memory size, and batch size — depending on the FPGA device and use case (e.g., B4096 vs B3136).

Scheduler, Load Engine, Compute Engine, Save Engine

The DPU pipeline includes the following key components:

1. **Scheduler:** Controls instruction execution and coordinates data movement among engines.
2. **Load Engine:** Reads input feature maps and weights from DDR memory and loads them into on-chip buffers.
3. **Compute Engine:** Performs matrix multiplications, convolutions, activations, and element-wise ops using arrays of DSPs and LUTs.
4. **Save Engine:** Writes processed output feature maps back to DDR.

This modular breakdown allows pipelining across layers and concurrent data processing.

Parallelism and Pipelining

The DPU supports both: - **Intra-layer parallelism:** Multiple processing elements handle different input/output channels simultaneously. - **Inter-layer pipelining:** Overlapping of load, compute, and save operations across consecutive layers improves throughput.

Batch processing and task-level parallelism are also supported when deploying multiple CUs on larger FPGAs.

Memory Interface and Resource Utilization

The DPU uses high-throughput AXI memory interfaces (HP ports) to interact with external DDR memory. Input/output tensors and weights are streamed through DMA.

Internally, it uses: - **BRAM:** For line buffers, feature maps, weights. - **DSPs:** For arithmetic-heavy operations (MACs). - **LUTs and FFs:** For control logic and lightweight arithmetic.

The resource utilization depends on DPU configuration and network size. Tools like Vitis Analyzer can visualize performance and utilization.

5.8.5 Complete Workflow Explanation

This section details the end-to-end deployment of deep learning models on FPGAs using the DPU-PYNQ framework.

Step 1: Train and Export Model

It begins by training deep learning model using a supported framework such as TensorFlow or PyTorch. Once trained:

- Export TensorFlow models as `.pb`
- Export PyTorch models as `.pt` or convert to `.onnx`

Make sure the model architecture is supported by the DPU.

Step 2: Quantize Model with Calibration

Use the Vitis AI Quantizer to convert the floating-point model to INT8 format:

- Provide a representative dataset (typically 100-1000 images)
- Perform post-training quantization to minimize accuracy loss
- Output is a quantized `.xmodel`

Step 3: Compile with Vitis AI Compiler

Compile the quantized model using ‘vai_c_xir‘ (for XIR-based workflow):

- Maps layers to DPU instructions
- Generates ‘model.xmodel‘, which contains the DPU-executable version of the network
- Must match the DPU architecture selected (e.g., B3136/B4096)

Step 4: Generate Bit-stream with Vitis

Create a hardware design in Vivado or Vitis:

- Integrate the DPU IP core
- Connect it with DMA, memory, and AXI interfaces
- Package the project into a bit-stream and export the overlay

This bit-stream will be used to configure the FPGA logic at runtime.

Step 5: Deploy on PYNQ Board

Transfer the following to the PYNQ board via SSH or SD card:

- DPU overlay (‘.bit‘, ‘.hwh‘, Python drivers)
- Quantized compiled model (‘.xmodel‘)
- Python inference script or Jupyter Notebook

Load the overlay dynamically using the PYNQ Python API.

Step 6: Run Inference Using Python and Overlays

Write a Python script that:

- Loads the DPU overlay (‘Overlay‘ class)
- Creates a DPU runner instance
- Performs image preprocessing (resize, normalize)
- Sends input tensor to DPU, triggers execution
- Collects and post-processes results (e.g., object detection boxes)

Inference can be performed in blocking or asynchronous mode. Results are displayed or logged using NumPy, OpenCV, or matplotlib.

5.8.6 Deployment on Ultra96-V2 (Case Study)

This section presents the deployment pipeline of the quantized YOLOv5-n model on the Ultra96-V2 board using the PYNQ framework. The entire process involves flashing the board with a compatible PYNQ image, setting up the required files, and executing a multiprocessing inference pipeline.

Board Setup

- Flashed the Ultra96-V2 board with a PYNQ-compatible Linux image.
- Connected to the board via USB serial and Ethernet.
- Launched the Jupyter Notebook interface by accessing `http://pynq:9090` in a browser.
- Ensured ‘pynq-dpu‘ is installed and functional by importing `DpuOverlay`.

File Structure on SD Card

The SD card is structured as follows:

```
dpu.bit                        # Compiled bitstream for DPU
model_dir/
    aa416_multiscale.xmodel    # Quantized YOLOv5-n model
img/
    voc_classes.txt            # Class names file
    agriauto_data/             # Input images
output/
    agriauto_outputs/          # Output images with bounding boxes
inference.py                   # Full inference pipeline script
```

Inference Pipeline Execution

The inference pipeline executes as follows:

1. Loads class names and model anchors.
2. Initializes DPU runner from `dpu.bit` and loads the quantized model.
3. Reads all images from `./img/agriautodata/`, applies preprocessing (resizing, normalization).
4. Uses `multiprocessing.Process` to overlap DPU execution and CPU postprocessing.
5. DPU worker feeds batches of images into the runner and collects feature maps.
6. CPU worker performs postprocessing:
    - Decodes feature maps.
    - Applies confidence thresholding and Non-Max Suppression (NMS).
    - Maps bounding boxes to original image coordinates.
    - Draws bounding boxes and class labels on the original images.
7. Final outputs are saved to `./output/agriauto`$_o utputs/$.

The core of the multiprocessing-based pipeline is shown in the following structure:

```
if __name__ == 'main':
    overlay = DpuOverlay('dpu.bit')
    overlay.load_model(MODEL_PATH)
```

```
runner = overlay.runner
...
p_dpu = Process(target=dpu_worker, ...)
p_cpu = Process(target=cpu_worker, ...)
p_dpu.start(); p_cpu.start()
p_dpu.join(); p_cpu.join()
```

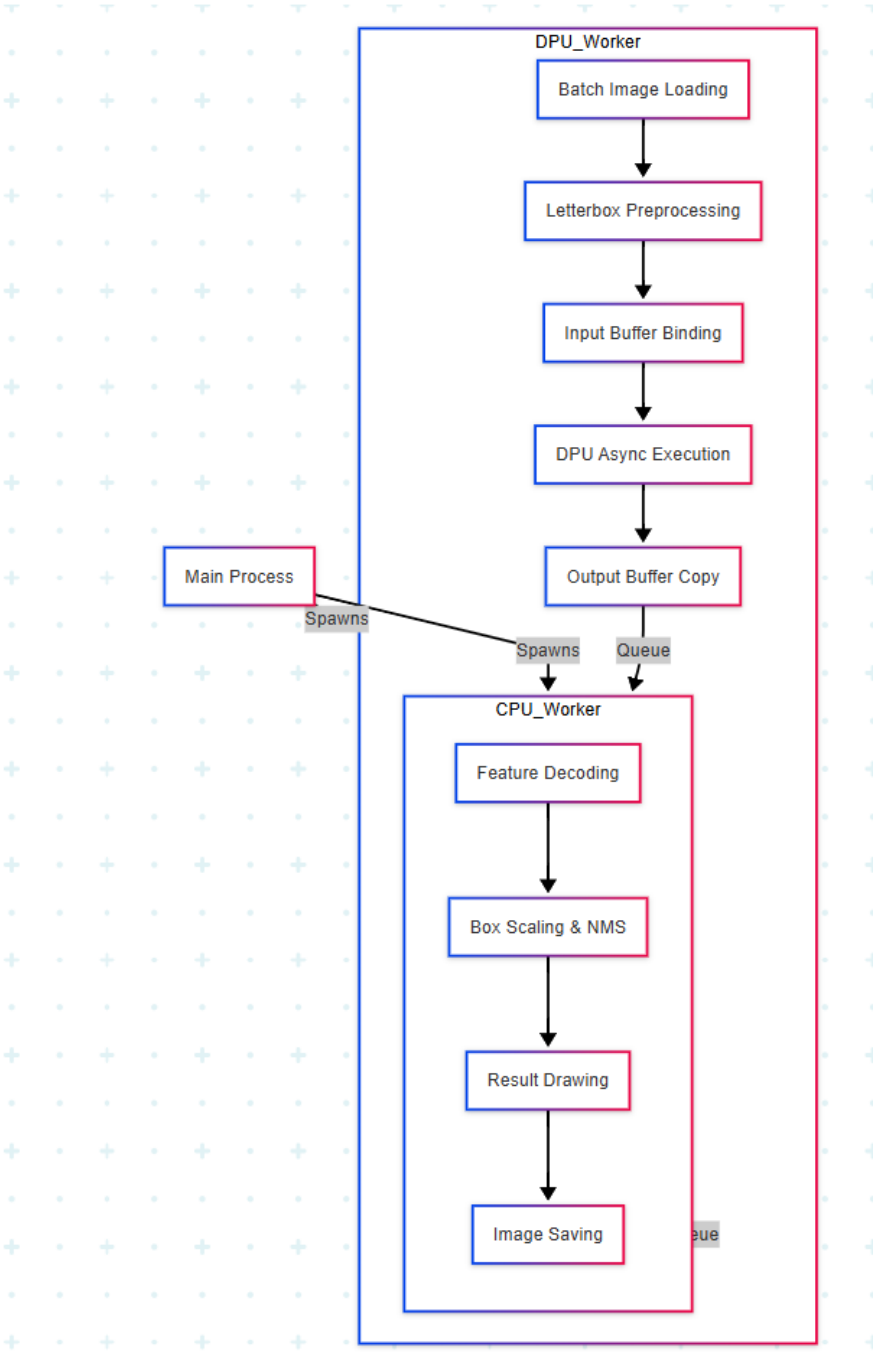


Figure 5.11: Ultra96v2 Inference Flow

This structure maximizes throughput by exploiting parallelism between image pre-

processing, inference, and postprocessing stages. The system demonstrates real-time capable inference performance for industrial-quality image data on an edge FPGA platform.

5.8.7 Conclusion

Deployment of a quantized YOLOv5-n model on Ultra96-V2 via DPU-PYNQ demonstrates effective edge AI acceleration using FPGA, achieving real-time, low-latency, low-power object detection through a parallel processing pipeline.

Benefits of using DPU-PYNQ:

- **Hardware Accelerated Inference and Energy Efficiency:** Offloading compute-intensive inference to the DPU significantly reduces inference time and CPU load. The Ultra96-V2's low power profile makes it ideal for embedded and mobile applications.
- **Flexibility:** The Python-based interface and Vitis AI tooling simplify integration with custom pipelines.
- **Parallelism:** Throughput and resource utilization are enhanced by the concurrent data flow made possible by the multiprocessing design.

Future Enhancements and Scalability:

- **Custom Postprocessing IP:** Implementing NMS and decoding logic as FPGA IP blocks using Vitis HLS can further offload CPU work.
- **Multi-DPU Support:** Scaling to multiple DPU cores for higher throughput in multi-camera systems.

This project provides a robust foundation for real-time, low-power edge AI systems applicable to industrial inspection, surveillance, and IoT deployments.

5.9 Discussion

The successful deployment of the quantized YOLOv5n model on FPGA hardware underscores several critical insights:

- **Effectiveness of Quantization Techniques:** Both PTQ and QAT are instrumental in optimizing neural networks for FPGA deployment. The choice between them should be informed by the specific model architecture and the availability of training data.

- **Role of Model Inspection:** Utilizing tools like the Vitis AI Inspector is vital to ensure model compatibility with FPGA hardware. This step identifies unsupported operators and guides necessary modifications to maximize DPU utilization.
- **Advantages of FPGA Deployment:** FPGAs offer a compelling solution for real-time deep learning applications, providing a balance between performance and energy efficiency. Their reconfigurable nature allows for tailored optimizations, aligning with the specific requirements of diverse industrial applications.

Chapter 6

Results

6.1 Overview

This chapter outlines the results obtained by deploying the model onto the Ultra96-V2 FPGA board. The performance is evaluated using mAP(mean Average Precision), Recall, Precision, FPS(Frames per Second) and power consumption in Watts.

6.2 Quantitative Performance Metrics

To thoroughly assess the deployment the model was deployed on 3 platforms: Ultra96-V2, RTX-3050 GPU, and 12th Gen i5(12450H). The specific metrics are outlined in Table 6.2

Table 6.2 Performance comparison between FPGA (Ultra96-V2), GPU (RTX-3050), and CPU platforms

Performance Metrics	Ultra96-V2	GPU (RTX-3050)	CPU
Frames per Second (FPS)	16	23.8	2.3
Power Consumption (Watts)	5.7	55	45
Mean Average Precision (mAP@0.5)	0.85	0.979	0.979
Precision	0.92	0.973	0.973
Recall	0.88	0.968	0.968

6.3 Detection Results Visualization

Inference outputs shown in Fig 6.1 and Fig 6.2 from the deployed model show detection of surface defects with varying confidence scores indicating the robust performance of the model to detect subtle and prominent defects under varying lighting and surface conditions.

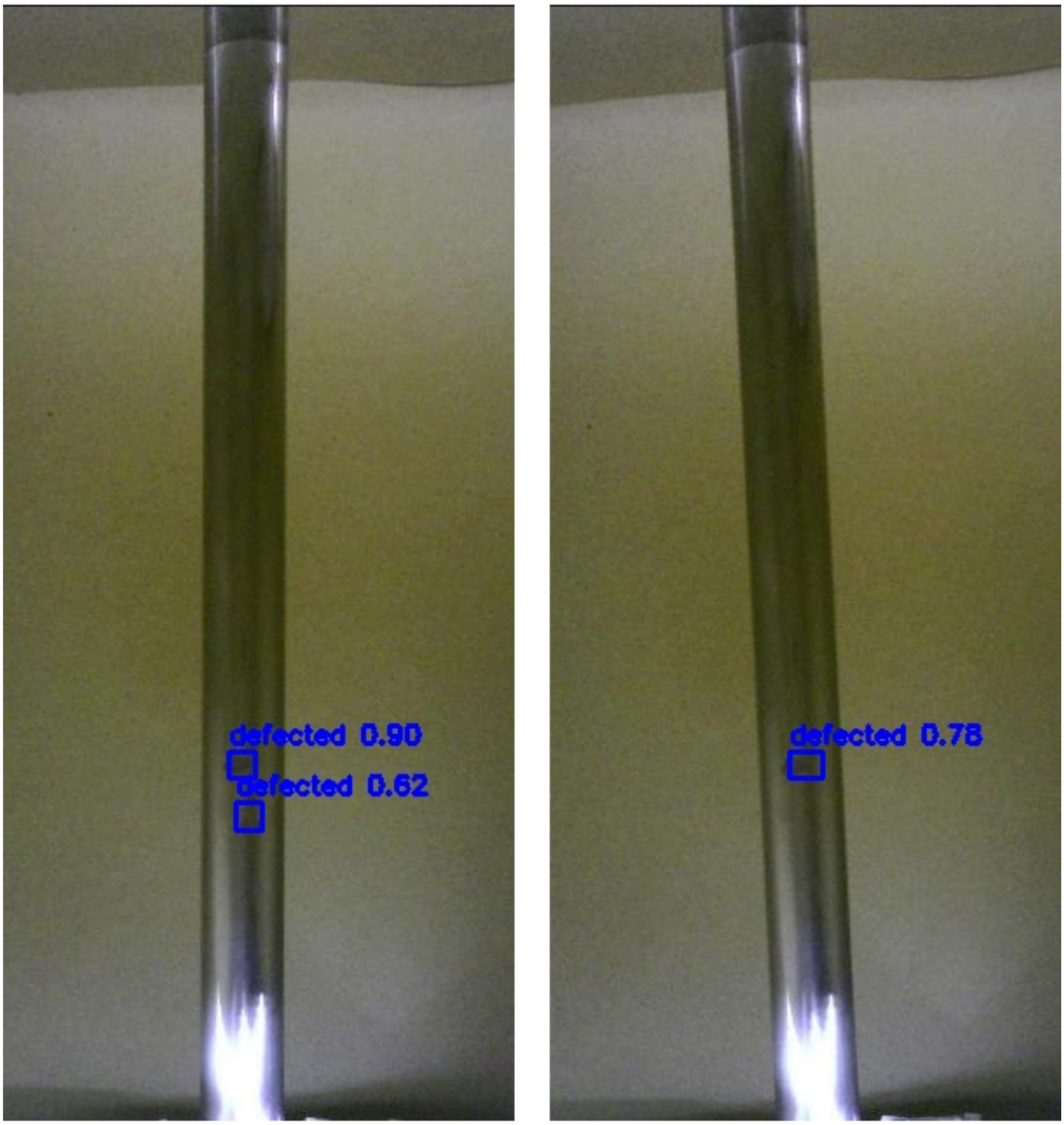


Figure 6.1: FPGA Inference.

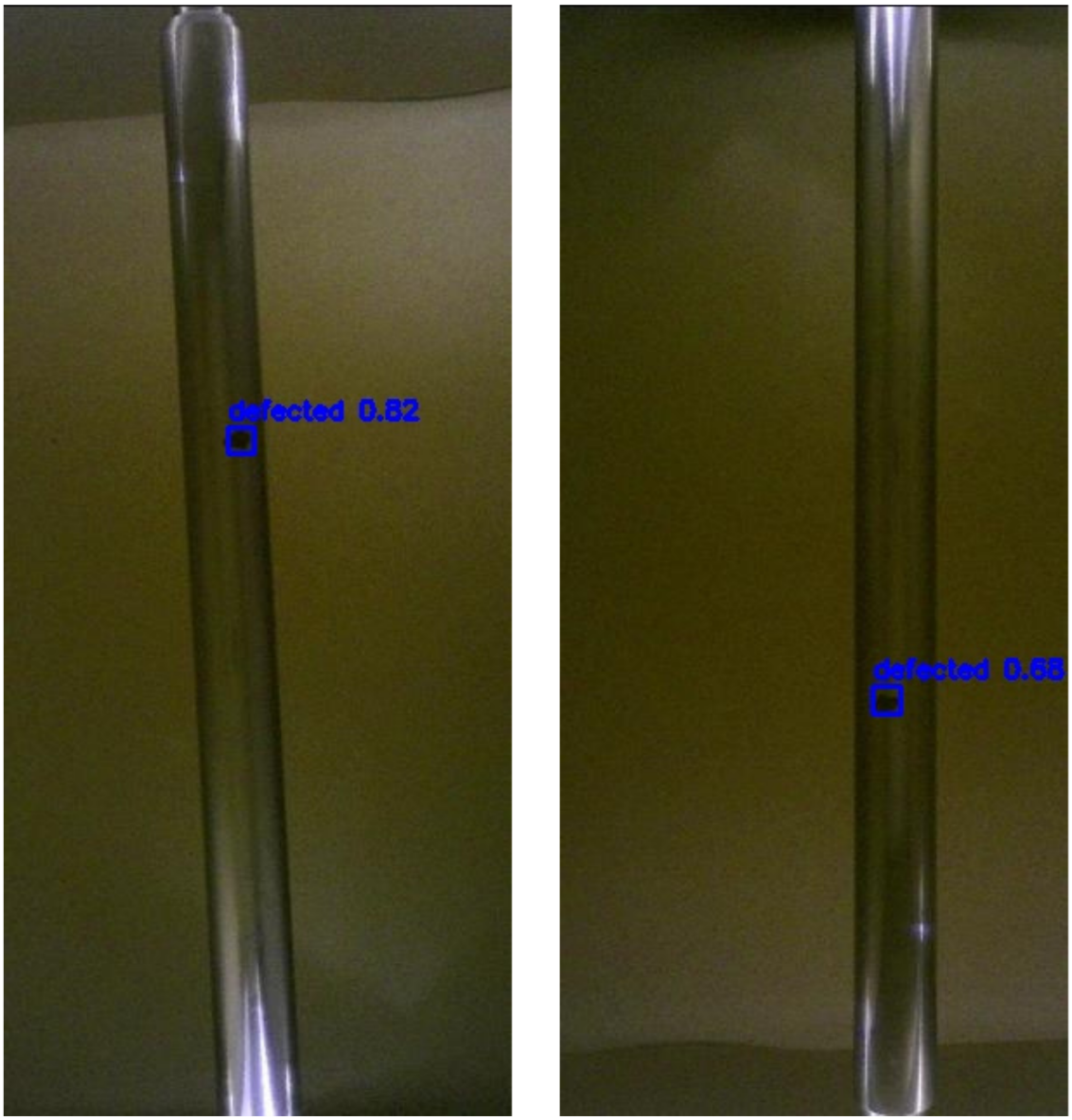


Figure 6.2: FPGA Inference.

6.4 Performance Optimization

- **Detection Rendering:** For each image, the detected bounding boxes are drawn with their confidence scores. These images are stored in an output folder.
- **FPS Evaluation:** The performance was measured both in terms of inference speed on the DPU and pre/post-processing speed on the CPU:
    1. DPU FPS shows the maximum speed achieved by FPGA inference.
    2. The FPS of the system, is hindered by the slowest stage of pre/post-processing.
- **Optimization Techniques:**
    1. Batch processing was used to increase the speed of image processing.
    2. The model input size was reduced to 416x416 to increase throughput.
    3. NMS(Non Max Suppression) was used to increase the post-processing speed.

6.5 Discussion

These results confirm that the deployment of the model on the Ultra96-V2 FPGA board is competitive in terms of FPS, power consumption, and accuracy and reliability. The minor reduction in accuracy, precision and recall is offset by improved efficiency, this solution is suited for industrial applications.

Chapter 7

User Interface

7.1 Overview

To enhance the usability and accessibility of our steel surface defect detection system, we developed a user-friendly, interactive dashboard titled **SteelSight**. The interface was built using React.js and Tailwind CSS to ensure a responsive and modern design. This dashboard acts as the bridge between the end user and the FPGA-accelerated inference engine, allowing real-time interaction with the defect detection system.

The primary objectives behind the design were:

- Enable intuitive access for uploading steel surface images.
- Facilitate FPGA-powered inference and visualize the results clearly.
- Present real-time performance metrics like precision, recall, and accuracy.

7.2 Features and Functionality

Figure 7.1: SteelSight Home Page: Overview and Project Summary

The home page provides a high-level overview of the project and introduces users to the detection flow. It showcases how the system works in three simple steps:

1. Upload an image of a steel rod or strip.
2. Run the defect detection on FPGA.
3. View annotated images with confidence scores.

Below the introductory section, real-world detections are displayed, showing sample outputs generated by the model.

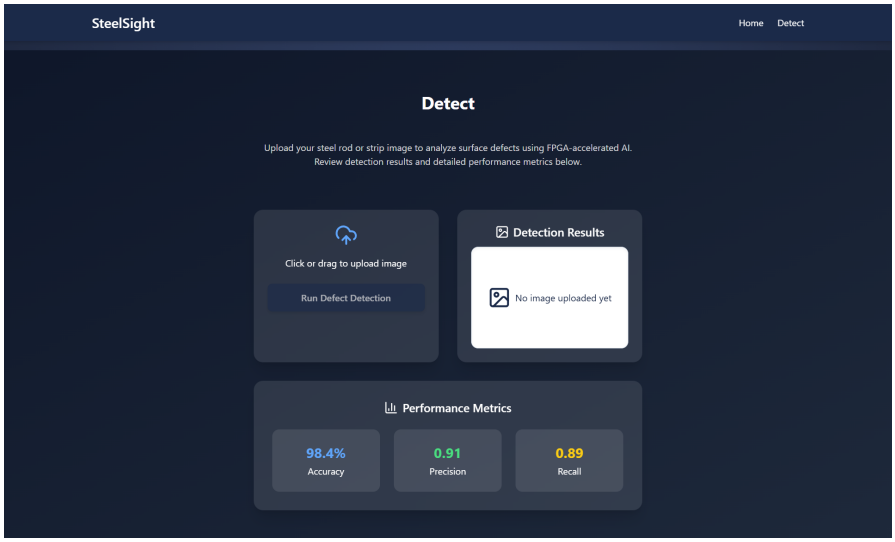


Figure 7.2: Detection Page: Upload Interface and Real-Time Metrics

The detection page enables direct interaction with the pipeline:

- Users can upload images using drag-and-drop or file picker.
- A detection result window displays annotated images with predicted defect classes.
- Real-time performance metrics including accuracy, precision, and recall are displayed for transparency.

7.3 Technology Stack

The dashboard leverages the following technologies:

- **React.js** – Modern JavaScript library for dynamic UI rendering.
- **Tailwind CSS** – Utility-first CSS framework for responsive and sleek design.
- **Framer Motion** – Animation library to enhance interactivity and transitions.
- **Integration with FPGA Backend** – The interface connects with backend APIs running on the FPGA, enabling low-latency inference and feedback.
- **Tripo AI (3D Visualization)** – Used to generate high-quality 3D assets and visual elements, contributing to a more dynamic and visually engaging user experience.

7.4 Impact and Role in the Pipeline

This UI layer not only makes the system accessible to non-technical users but also plays a crucial role in:

- Demonstrating the practicality of deploying ML models in industrial settings.
- Allowing immediate visual validation of inference accuracy.
- Acting as a foundation for future upgrades such as real-time camera stream processing.

Overall, the SteelSight dashboard transforms the technical backend into a usable and appealing product ready for industrial trials.

Chapter 8

Conclusion and Future Enhancements

8.1 Conclusion

This project successfully delivers a complete machine vision pipeline for defect detection on steel surfaces, optimized for industrial application. By leveraging deep learning and deploying the model on FPGA hardware, we address key industrial needs—fast processing, energy efficiency, and scalability.

From initial experimentation with NEU-DET to combining it with GC10-DET and ultimately incorporating real-world images from Agriauto Industries, the dataset strategy evolved to reflect real operational challenges. A lightweight yet effective YOLOv5n model was selected for its balance between inference speed and accuracy. The model was quantized to INT8 and deployed using the Vitis AI toolchain onto the Avnet Ultra96-V2 FPGA platform.

The system demonstrated:

- Robust detection performance on diverse steel surface images.
- Effective inference capabilities post-quantization.
- Energy-efficient execution using FPGA-based acceleration.
- A functional and user-friendly interface for testing and visualization.

These components collectively establish a strong technical foundation for an industrial-grade defect detection solution. With the entire pipeline—from data acquisition to deployment—designed and evaluated, the project meets its intended goals and paves the way for further enhancements and real-world scalability.

8.2 Future Enhancements

Although the current system delivers reliable and optimized performance within the scope of our project, its full potential in real-world industrial environments can be further unlocked through targeted enhancements. These improvements aim to bridge the gap between a successful prototype and a robust, production-ready solution—ensuring the system performs consistently under dynamic operational conditions, integrates seamlessly with existing workflows, and scales efficiently across various industrial settings.

8.3 Real-time Defect Detection with Camera Integration

While the current system is functional and optimized, several strategic improvements have been identified to enhance practicality, robustness, and industrial readiness. One such enhancement is integrating real-time image acquisition using compatible camera modules.

To enable real-time defect detection, suitable camera modules that interface with the Avnet Ultra96-V2 board are essential. Below are two recommended options:

8.3.1 Dual AR0144 IAS Sensor Modules

The AR0144 is a compact, monochrome camera sensor designed for machine vision applications. It features:

- Resolution: 1 MP
- Sensor Type: Monochrome global shutter
- Compatibility: Designed to integrate with Ultra96-V2 via the 96Boards Onsemi Dual Camera Mezzanine board.
- Ideal for high-speed, low-latency vision tasks.

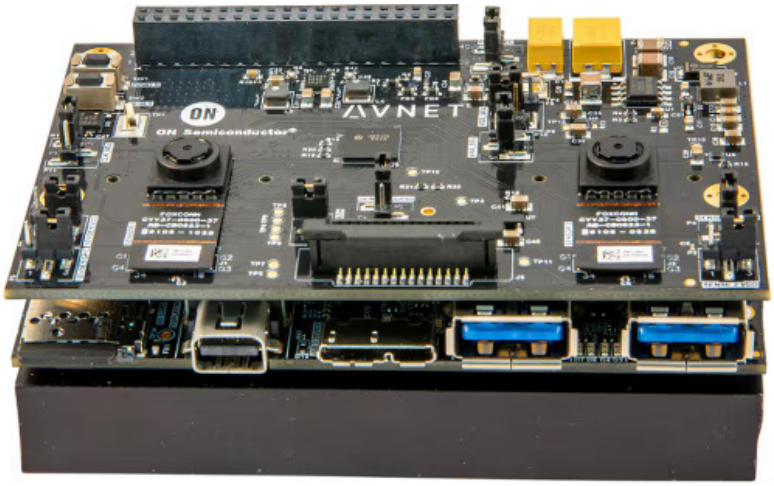


Figure 8.1: OnSemi Dual Camera Mezzanine Board for 96Boards

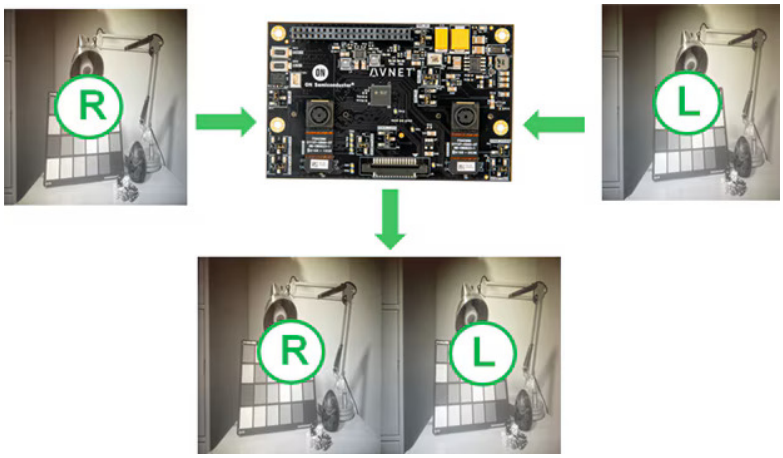


Figure 8.2: Dual AR0144 IAS Sensor Modules

8.3.2 Single AR1335 IAS Sensor Module

The AR1335 is a compact, 4k color camera sensor designed for machine vision applications. It features:

- Resolution: 16 MP
- Sensor Type: Rolling shutter auto-focus
- Compatibility: Designed to integrate with Ultra96-V2 via the 96Boards Onsemi Dual Camera Mezzanine board.
- Ideal for high-speed, low-latency vision tasks.

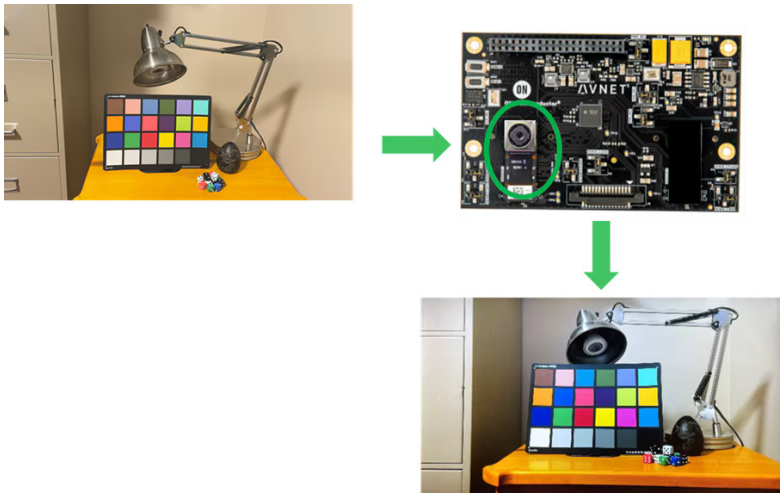


Figure 8.3: Single AR1335 IAS Sensor Modules

8.3.3 Enhanced Energy Efficiency with Advanced FPGA Platform

Although the Avnet Ultra96-V2 platform was sufficient for prototype deployment, exploring more advanced FPGA boards—such as the Xilinx ZCU102 or ZCU104—could further reduce power consumption while supporting higher-resolution inputs or larger models. This move would:

- Improve energy efficiency in continuous operations.
- Enable broader deployment across resource-constrained environments.

8.3.4 Expanded Defect Classification and Model Refinement

Currently, the model performs binary classification. For more granular insights, future versions can include:

- Multi-class detection of surface anomalies (e.g., scratches, cracks, dents, inclusions).
- Advanced model architectures with higher resolution input support.
- Domain adaptation to ensure robustness across different steel products.
- Combine YOLOv5 with Domain-Adversarial Neural Networks to improve defect detection accuracy across varying environments and reduce sensitivity to domain shifts.

8.3.5 Backend Integration and Industrial Dashboard

To support monitoring, tracking, and continuous improvement:

- A backend server can be integrated to store image logs, defect results, and timestamps.
- An analytics dashboard can visualize trends and production metrics.
- Integration with existing manufacturing systems (e.g., SCADA) can close the loop for automated defect rejection.

8.4 Closing Remarks

This Final Year Project establishes a viable, scalable, and energy-aware defect detection pipeline that meets the accuracy and speed requirements of industrial applications. While the current system functions reliably, the planned enhancements—real-time camera integration, hardware upgrades, and advanced classification—will position it for seamless adoption in smart manufacturing setups.

The progress made during this project not only reflects technical achievement but also sets the stage for ongoing innovation and real-world impact in automated quality assurance systems.

Bibliography

[1] Rashed Al Amin, Mehrab Hasan, Veit Wiese, and Roman Obermaisser. Fpga-based real-time object detection and classification system using yolo for edge computing. *IEEE Access*, 12:73268–73278, 2024.

[2] Chun Bao, Tao Xie, Wenbin Feng, Le Chang, and Chongchong Yu. A power-efficient optimizing framework fpga accelerator based on winograd for yolo. *IEEE Access*, 8:94307–94317, 2020.

[3] Sebastian Bunda, Nikolaos Alachiotis, and Luuk Spreeuwers. Accelerating selective sweep detection using amd deep learning processing units and vitis ai. In *45th Symposium on Information Theory and Signal Processing (SITB 2025)*, pages 8–11, May 2025. 45th Symposium on Information Theory and Signal Processing, SITB 2025, SITB 2025 ; Conference date: 19-05-2025 Through 20-05-2025.

[4] Zhi Cheng, Liping Gao, Yu Wang, Zaohui Deng, and Yin Tao. Ec-yolo: Effectual detection model for steel strip surface defects based on yolo-v5. *IEEE Access*, 12:62765–62778, 2024.

[5] Shiv Ram Dubey, Satish Kumar Singh, and Bidyut Baran Chaudhuri. Activation functions in deep learning: A comprehensive survey and benchmark. *Neurocomputing*, 503:92–108, 2022.

[6] Yuta Fukuda, Kota Yoshida, and Takeshi Fujino. Evaluation of model quantization method on vitis-ai for mitigating adversarial examples. *IEEE Access*, 11:87200–87209, 2023.

[7] Lokesh Heda and Parul Sahare. Performance evaluation of yolov3, yolov4 and yolov5 for real-time human detection. In *2023 2nd International Conference on Paradigm Shifts in Communications Embedded Systems, Machine Learning and Signal Processing (PCEMS)*, pages 1–6, 2023.

[8] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. pages 2704–2713, 06 2018.

[9] Aicha Khalfaoui, Abdelmajid Badri, and Ilham EL Mourabit. Comparative study of yolov3 and yolov5's performances for real-time person detection. In *2022 2nd International Conference on Innovative Research in Applied Science, Engineering and Technology (IRASET)*, pages 1–5, 2022.

[10] Raghuraman Krishnamoorthi. Quantizing deep convolutional networks for efficient inference: A whitepaper. *CoRR*, abs/1806.08342, 2018.

[11] Zhengdong Li, Frederick Hong, and C. Yue. Fpga-based acceleration of neural network for image classification using vitis ai, 12 2024.

[12] Zhengdong Li, Frederick Ziyang Hong, and C. Patrick Yue. Fpga-based acceleration of neural network for image classification using vitis ai, 2024.

[13] Junjiang Liu, Mingyu Mao, Jianlong Gao, Jiangtao Bai, and Dandan Sun. Hardware-accelerated yolov5 based on mpsoc. *Journal of Physics: Conference Series*, 2732(1):012013, mar 2024.

[14] Michal Machura, Michal Danilowicz, and Tomasz Kryjak. Embedded object detection with custom littlenet, finn and vitis ai dcnn accelerators. *Journal of Low Power Electronics and Applications*, 12(2), 2022.

[15] Duy-Linh Nguyen, Xuan-Thuy Vo, Adri Priadana, and Kang-Hyun Jo. Vehicle detector based on improved yolov5 architecture for traffic management and control systems. In *IECON 2023- 49th Annual Conference of the IEEE Industrial Electronics Society*, pages 1–7, 2023.

[16] Daniel Pohl, Dr.-Ing. Birgit Vogel-Heuser, Marius Krüger, and Markus Echtler. Quantization effects of deep neural networks on a fpga platform. In *2024 IEEE 7th International Conference on Industrial Cyber-Physical Systems (ICPS)*, pages 1–8, 2024.

[17] Yunhui Qu and Jintao Wang. Design of low-power yolo accelerator based on fpga. pages 1132–1137, 03 2024.

[18] Yunhui Qu and Jintao Wang. Design of low-power yolo accelerator based on fpga. In *2024 7th International Conference on Advanced Algorithms and Control Engineering (ICAACE)*, pages 1132–1137, 2024.

[19] Shankar R and Muthulakshmi M. Comparing yolov3, yolov5 yolov7 architectures for underwater marine creatures detection. In *2023 International Conference on Computational Intelligence and Knowledge Economy (ICCIKE)*, pages 25–30, 2023.

[20] Jonathan Sanderson and Syed Rafay Hasan. System integration of xilinx dpu and hdmi for real-time inference in pynq environment with image enhancement, 2023.

[21] Harshil Shah and Vaishnavi Patil. Metal surface defect detection using object detection models. In *2023 International Conference on Intelligent and Innovative Technologies in Computing, Electrical and Electronics (IITCEE)*, pages 135–139, 2023.

[22] Kaiping Sun, Jianping Xing, Yu Bai, Shuzhen Huang, Siqi He, and Peng Sun. Implementation of image processing model based on vitis-ai architecture. In *Proceedings of the 2024 International Conference on Image Processing, Multimedia Technology and Maching Learning*, IPMML '24, page 18–23, New York, NY, USA, 2025. Association for Computing Machinery.

[23] Akihiko Ushiroyama, Minoru Watanabe, Nobuya Watanabe, and Akira Nagoya. Convolutional neural network implementations using vitis ai. In *2022 IEEE 12th Annual Computing and Communication Workshop and Conference (CCWC)*, pages 0365–0371, 2022.

[24] Chenglong Wang, Ziran Zhou, and Zhiming Chen. An enhanced yolov4 model with self-dependent attentive fusion and component randomized mosaic augmentation for metal surface defect detection. *IEEE Access*, 10:97758–97766, 2022.

[25] Lin Wang, Tianyong Ao, Le Fu, Jian Liu, Yang Liu, and Yi Zhou. Design of a yolo model accelerator based on pynq architecture. In *2022 International Conference on Machine Learning and Intelligent Systems Engineering (MLISE)*, pages 15–18, 2022.

[26] Zhichen Wang, Hengyi Li, Xuebin Yue, and Lin Meng. Briefly analysis about cnn accelerator based on fpga. *Procedia Computer Science*, 202:277–282, 2022. International Conference on Identification, Information and Knowledge in the internet of Things, 2021.

[27] Gaomiao Xu, Wei Zhao, Zhouqi Ren, Zhenjia Chen, and Jiabao Gao. Design and implementation of the high-performance yolo accelerator based on zynq fpga. In *2024 3rd International Conference on Electronics and Information Technology (EIT)*, pages 246–250, 2024.

[28] Kaijun Yang and Tao Chen. Lightweight surface defect detection algorithm based on improved yolov5. In *2024 5th International Conference on Mechatronics Technology and Intelligent Manufacturing (ICMTIM)*, pages 798–802, 2024.

[29] Jiangli Yu, Xiangnan Shi, Wenhai Wang, and Yunchang Zheng. Lcg-yolo: A real-time surface defect detection method for metal components. *IEEE Access*, 12:41436–41451, 2024.

[30] Zhichao Zhang, M. A. MAHMUD, and Abbas Kouzani. Resource-constrained fpga implementation of yolov2. *Neural Computing and Applications*, 34:1–18, 05 2022.

[31] Fangbo Zhou, Huailin Zhao, and Zhen Nie. Safety helmet detection based on yolov5. In *2021 IEEE International Conference on Power Electronics, Computer Applications (ICPECA)*, pages 6–11, 2021.